KUMASI TECHNICAL UNIVERSITY
FACULTY OF APPLIED SCIENCES AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

DESIGN AND IMPLEMENTATION OF A WEB-BASED FARM MANAGEMENT SYSTEM

(A CASE STUDY OF GREENACRES FARMS, EJISU — ASHANTI REGION)

BY

BOATEMAA KYEREWAA JANTUAH

INDEX NUMBER: 052420760094

A PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE,
KUMASI TECHNICAL UNIVERSITY,
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF
DIPLOMA IN INFORMATION TECHNOLOGY (DIT)

AUGUST, 2026

DECLARATION

I hereby declare that this project report is the result of my own original work carried out under supervision, and that to the best of my knowledge it contains no material previously published by another person, nor material which has been accepted for the award of any other degree or diploma of this university or any other institution, except where due acknowledgement has been made in the text.

All sources of information consulted in the course of this work have been duly cited and referenced.

BOATEMAA KYEREWAA JANTUAH (052420760094)

Student

Signature: ..... Date: .....

SUPERVISOR

Name: .....

Signature: ..... Date: .....

HEAD OF DEPARTMENT

Name: .....

Signature: ..... Date: .....

DEDICATION

This work is dedicated to my parents, whose sacrifice and constant encouragement made this programme possible, and to the farming families of the Ashanti Region whose daily labour feeds the nation and inspired the subject of this study.

And to every student who will pick this report up after me — may it be of use.

ACKNOWLEDGEMENT

First and foremost, I give thanks to the Almighty God for the gift of health, strength and understanding throughout the period of this programme.

My sincere gratitude goes to my project supervisor, whose patient guidance, constructive criticism and technical direction shaped this work from a rough idea into a working system. The time given to reading drafts and questioning design decisions improved both the software and the report considerably.

I am grateful to the Head and the entire teaching staff of the Department of Computer Science, Kumasi Technical University, for the foundation laid over the course of this programme; the lessons in database design, programming and systems analysis are the direct basis of the work presented here.

I acknowledge the farm owners, managers and workers who gave their time during the fact-finding stage; their willingness to explain how records are actually kept, rather than how they are supposed to be kept, is what grounded this system in reality. Finally, I thank my family and colleagues for their support and understanding during the long hours this project demanded.

ABSTRACT

Small and medium-scale farms in Ghana continue to depend on manual, paper-based record keeping. Livestock registers, feed purchases, planting dates, wage payments and sales records are kept in separate notebooks that are rarely reconciled with one another. The consequence is that information which already exists on the farm cannot be turned into knowledge: the farmer cannot readily say which crop was profitable last season, which animal is due for vaccination, or whether the store is about to run out of feed. Records are also vulnerable to loss, damage and arithmetic error, and there is no reliable trail of who recorded what.

This project designed and implemented a web-based Farm Management System that consolidates every farm enterprise into a single relational database and presents the result as actionable management information. It was developed using PHP 8 and MySQL on the XAMPP platform following an iterative and incremental model, with requirements established through interviews, document analysis and direct observation at a mixed farm at Ejisu in the Ashanti Region. The delivered system comprises sixteen normalised database tables and seventeen functional modules covering livestock, animal health, production, crops, fields, harvests, inventory, suppliers, tasks, finance, employees, reporting, user administration, auditing and configuration. Role-based access control distinguishes administrator, manager and worker, and is enforced on the server rather than merely by hiding controls in the interface. Security measures include prepared statements, bcrypt password hashing, cross-site request forgery tokens, output escaping and session management, while stock balances are maintained through an auditable movement ledger applied within database transactions.

A deliberate constraint was that the system must operate without an internet connection, since connectivity at farm locations is intermittent. This ruled out the content delivery networks on which common charting and icon libraries depend, so both the charting engine and the icon set were written from scratch, leaving no runtime dependency of any kind. Twenty-four documented test cases were executed and all passed, including one confirming that a user holding only the worker role cannot create records by submitting a forged request directly to the server; testing also exposed two genuine defects, which were traced, corrected and documented. The results demonstrate that a useful farm management information system can be built on free, locally hosted technology, and that the offline constraint produced a more self-contained and more defensible engineering outcome.

Keywords: *farm management information system, relational database design, role-based access control, PHP, MySQL, XAMPP, agricultural record keeping, offline web application.*

TABLE OF CONTENTS

LIST OF FIGURES

The plates listed below reproduce the wider design diagrams and principal screens at full page size, in landscape orientation, in Appendix A.

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DBMS	Database Management System
DFD	Data Flow Diagram
DIT	Diploma in Information Technology
ERD	Entity Relationship Diagram
FMIS	Farm Management Information System
GHS	Ghana Cedi
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ICT	Information and Communication Technology
KPI	Key Performance Indicator
PDO	PHP Data Objects
PHP	PHP: Hypertext Preprocessor
RBAC	Role-Based Access Control
SDLC	Software Development Life Cycle
SQL	Structured Query Language
SVG	Scalable Vector Graphics
UML	Unified Modelling Language
XAMPP	Cross-platform, Apache, MySQL, PHP, Perl
XSS	Cross-Site Scripting
3NF	Third Normal Form

CHAPTER ONE
INTRODUCTION

1.1 Introduction

Agriculture remains the backbone of the Ghanaian economy. It employs a substantial proportion of the working population, supplies the raw material for much of the country's agro-processing industry, and underpins national food security. Despite this importance, the great majority of farms in the country are small to medium in scale and are managed with very little formal information support. Decisions about what to plant, when to sell, how much feed to buy and which animal to cull are frequently taken from memory or from figures scattered across several notebooks.

Information technology has transformed record keeping in banking, retail, health and education, yet its penetration into everyday farm management in the sub-region remains shallow. Where computerised solutions do exist, they are often priced in foreign currency, sold on a subscription basis, and designed on the assumption of permanent internet connectivity — three conditions that do not hold for the typical farm in the Ashanti Region. This project responds to that gap. It presents the analysis, design, implementation and testing of a web-based Farm Management System capable of running on a single computer using freely available software, without an internet connection and without recurring cost, consolidating livestock, crop, inventory, personnel and financial records into one relational database and converting them into the operational information a farm manager actually needs.

1.2 Background of the Study

A farm, however modest, is a business made up of several interacting enterprises. A mixed farm of the kind studied in this project may simultaneously run a dairy herd, a small ruminant flock, a poultry unit, several fields of arable crops, a store of inputs, and a payroll. Each of these generates records, and each record influences decisions taken elsewhere. The cost of feed bought for the poultry unit, for example, is meaningless in isolation; it becomes information only when set against the eggs that unit produced and the price at which they were sold.

In practice, however, these records are kept separately. A livestock register may be maintained in one exercise book, purchases in another, and sales on loose sheets that are frequently mislaid. Reconciliation, where it happens at all, occurs at the end of a season and is performed by hand with a calculator. The arithmetic burden is significant, the risk of error is high, and the exercise is so laborious that it is often abandoned.

The case study for this project is GreenAcres Farms, a mixed farming operation situated at Ejisu in the Ashanti Region of Ghana. The farm keeps cattle, goats, sheep, pigs and poultry, cultivates maize, rice, cassava, plantain and vegetables across roughly fifty-four acres of land in six parcels, employs eight members of staff, and maintains a store of feed, seed, fertiliser, agrochemicals, veterinary supplies, tools and fuel. It is, in other words, exactly the scale of

enterprise at which manual record keeping begins to break down: large enough to generate a great deal of data, but not large enough to justify an expensive commercial system or a dedicated records officer.

1.3 Statement of the Problem

The central problem this project addresses is that the manual, paper-based record keeping used on small and medium farms fails to convert the data the farm already produces into information the farm can act upon. This general failure manifests in five specific ways:

1. Fragmentation. Records for livestock, crops, stores, staff and money are held in physically separate books and are never brought together, so cross-enterprise analysis is impossible.
2. Absence of timely alerting. Conditions that require action — stock falling below a safe level, a treatment becoming due, a task passing its deadline — are only discovered after the loss has been incurred.
3. Analytical poverty. Profitability by crop, by field or by enterprise is not calculated, because the volume of manual arithmetic involved is prohibitive.
4. Insecurity and fragility of the record. Paper is lost, damaged by water and insects, and easily altered. There is no backup and no version of the truth other than the book itself.
5. Absence of accountability. There is no record of who entered a figure, who altered it, or when. Disputes over stock issues or sales receipts cannot be resolved from the record.

Taken together these deficiencies mean the farm operates with a substantial information deficit, and that management decisions are made on impression rather than evidence.

1.4 Aim of the Study

The aim of this study is to design and implement a secure, multi-user, web-based Farm Management System that consolidates all farm records into a single relational database and presents them as timely, accurate and actionable management information, using technology that is freely available and capable of operating without an internet connection.

1.5 Objectives of the Study

In order to achieve the stated aim, the following specific objectives were pursued:

1. To design a normalised relational database capable of representing every enterprise carried on by a mixed farm.
2. To implement secure multi-user access with role-based authorisation appropriate to the responsibilities of an owner, a manager and a field worker.
3. To develop complete record management functions for livestock, animal health, production, crops, fields, harvests, inventory, suppliers, tasks, finance and personnel.
4. To generate automatic operational alerts for low stock, overdue tasks and due veterinary treatments.

5. To produce analytical dashboards and printable management reports covering the whole operation.
6. To maintain a complete and tamper-evident audit trail of all activity carried out within the system.
7. To test the resulting system for functional correctness, data integrity and resistance to common web security attacks.

1.6 Research Questions

The study was guided by the following questions:

1. What records does a mixed farm generate, and what are the relationships between them?
2. How can these records be represented in a relational schema that avoids redundancy and preserves integrity?
3. What division of authority between farm roles is appropriate, and how may it be enforced reliably?
4. Which items of management information are most valuable to a farm manager, and how should they be presented?
5. Can a system of this kind be delivered on freely available technology without an internet connection?

1.7 Scope and Delimitation of the Study

The study covers the analysis, design, implementation and testing of a farm management system for a single farm enterprise operated by multiple users on a local computer or local area network. Functionally, the scope embraces livestock and herd health, daily production, crop and field management, harvest recording, inventory and supplier management, task assignment, income and expenditure recording, employee records, reporting, user administration and system auditing.

The following are expressly outside the scope of this work and are identified in Chapter Five as opportunities for further development:

- Native mobile applications for Android or iOS.
- Short message service or electronic mail gateways for external alerting.
- Global positioning system mapping, remote sensing or internet-of-things sensor integration.
- Statutory financial accounting, taxation computation and payroll deduction.
- Multi-farm or cooperative tenancy, in which several independent farms share one installation.
- Deployment to a public internet host, with the additional security hardening that would entail.

1.8 Significance of the Study

The significance of this study operates at three levels. For the case study farm, the system replaces a fragmented paper process with a single reliable source of truth, gives the owner same-day visibility of profitability, and reduces measurable losses caused by stock-outs and missed veterinary schedules.

For the wider farming community, the work demonstrates that a useful farm management information system need not be expensive or dependent on connectivity. Because the system is built entirely on free software and requires no internet access at run time, it can be replicated at any farm possessing a single ordinary computer.

For the academic community and for future students, the report documents a complete software development life cycle applied to a real problem: requirements elicited from practitioners, a normalised database designed from those requirements, a layered implementation, and a documented programme of testing that found and corrected genuine defects. The source code and database schema accompanying this report are available for study, extension and reuse.

1.9 Limitations of the Study

Several limitations should be acknowledged. First, the system was developed and evaluated principally against the records and workflows of a single farm; although care was taken to keep the design general, farms with substantially different enterprises may require schema extension. Second, the period available for the study did not permit a full season of live parallel running alongside the manual system, so the quantitative benefit is projected from testing rather than measured over a complete production cycle. Third, alerting is delivered within the application interface only, which requires a user to open the system in order to see it. Fourth, the financial module provides management accounting suitable for internal decision making, and is not a substitute for statutory accounts prepared by a qualified accountant.

1.10 Definition of Key Terms

The following terms are used throughout this report with the meanings given below.

Term	Meaning
Farm Management Information System	A computer-based system that collects, stores, processes and reports data arising from farm operations to support management decisions.
Normalisation	The process of organising the columns and tables of a relational database so as to minimise redundancy and avoid update anomalies.
Role-based access control	A method of restricting system access whereby permissions are attached to roles, and users acquire permissions by being assigned to a role.
Prepared statement	A pre-compiled SQL statement into which values are passed as parameters rather than being concatenated into the query text, thereby preventing SQL injection.
Hashing	A one-way transformation of data, used here so that stored passwords cannot be recovered even by an administrator with full database access.

Term	Meaning
Audit trail	A chronological record of system activity sufficient to reconstruct who performed which operation and when.
XAMPP	A free, cross-platform software bundle comprising the Apache web server, the MySQL/MariaDB database server, and the PHP and Perl interpreters.

1.11 Organisation of the Report

This report is organised into five chapters. Chapter One has introduced the study, established its background, stated the problem, and defined the aim, objectives and scope of the work.

Chapter Two reviews the relevant literature. It examines the concept of farm management, the state of agricultural record keeping, the characteristics of farm management information systems, and the technologies applicable to their construction, concluding with an analysis of the gap this project fills.

Chapter Three presents the system analysis and design. It describes the development methodology and fact-finding techniques employed, analyses the existing manual system, states the functional and non-functional requirements, and presents the design of the proposed system through an architectural diagram, a use case diagram, an entity relationship diagram, a data dictionary, process flowcharts and data flow diagrams.

Chapter Four documents the implementation and testing of the system. It describes the development environment, presents the implemented modules with accompanying screenshots, explains the principal algorithms and security measures, and reports the results of unit, integration, system and security testing, including the defects discovered and their resolution.

Chapter Five summarises the work, assesses the extent to which each objective was achieved, draws conclusions, and offers recommendations for future development. References and appendices follow, the latter containing full-size interface screenshots, the database schema, extracts of the source code and the installation guide.

CHAPTER TWO
LITERATURE REVIEW

2.1 Introduction

This chapter reviews the body of knowledge relevant to the design of a farm management information system. It begins by establishing what farm management is understood to involve and why information is central to it. It then considers the state of agricultural record keeping, the nature and classification of farm management information systems, and a comparative review of solutions currently available. The chapter goes on to examine the specific technologies and theoretical principles applied in this project — relational database theory and normalisation, three-tier application architecture, web application security and interface design — and closes by identifying the gap that the present work addresses.

2.2 The Concept of Farm Management

Farm management is conventionally defined as the application of business and economic principles to the organisation and operation of a farm as a productive enterprise. It concerns the allocation of scarce resources — land, labour, capital and management attention — among competing enterprises in order to achieve the objectives of the farm household, which are typically some combination of profit, food security and risk reduction.

Three characteristics distinguish farm management from the management of other small businesses. The first is biological lag: decisions taken today produce their outcome only after a growing or gestation period during which they cannot easily be reversed. The second is exposure to weather and disease, which introduces variance that management can mitigate but not eliminate. The third is the joint nature of production, in which enterprises share the same labour force, store and machinery, so that the true cost of any one can only be established by apportioning shared inputs.

Each characteristic increases, rather than reduces, the value of good records. Because decisions cannot be reversed they must be taken well the first time, which requires evidence; because variance is high a single season is a poor guide and trends are needed; and because production is joint, the profitability of an enterprise must be computed from records of shared input use rather than observed directly.

2.3 Record Keeping in Agricultural Enterprises

The literature on agricultural extension consistently identifies record keeping as a determinant of farm profitability, and equally consistently reports that it is poorly practised on small farms. The records a mixed farm ought to keep fall into five broad categories: physical inventories of land, livestock and equipment; production records of yields and outputs; input records of purchases and their use; labour records; and financial records of receipts and payments.

Studies of smallholder practice repeatedly find that where records are kept at all, they are kept as transaction lists rather than as an integrated system. Purchases are noted because money changed hands and the farmer wishes to remember it; production is noted sporadically; and the two are never brought into relation with one another. The consequence is that the farmer possesses data but not information, and cannot answer management questions from the record.

Four barriers are commonly reported: limited literacy and numeracy among some operators; the perception that record keeping competes with field work; the absence of a structure telling the farmer what to record; and the arithmetic burden of turning raw records into summaries. A computerised system addresses the last two directly — it supplies the structure through its data entry forms, and removes the arithmetic entirely.

A comparison of manual and computerised records across the dimensions that matter to a working farm makes the case plain. Manual records are cheap to begin and need no equipment, but they scale badly, cannot be searched or aggregated without effort, admit no concurrent access, provide no alerting, and offer no protection against loss or unauthorised alteration. Computerised records reverse each of these properties at the cost of an initial investment in equipment and training. The relevant question for a small farm is therefore not whether computerisation is superior in principle, but whether the cost of entry can be brought low enough — a question this project answers directly by building on software that costs nothing and hardware such farms already possess.

2.4 Farm Management Information Systems

A farm management information system may be defined as a planned system for the collection, processing, storage and dissemination of data in the form required to carry out the operations and management functions of a farm. Such systems are commonly classified by their functional coverage into three groups.

Enterprise-specific systems address a single farm activity in depth — herd management software for dairy operations, for instance — offering rich functionality within their domain but unable to answer questions that cross enterprise boundaries. Integrated management systems, of which the present work is an example, cover the whole farm at a level of detail sufficient for management, deliberately trading depth in any one enterprise for the ability to relate enterprises to one another.

This project situates itself firmly in the second category. The judgement underlying that choice is that for a farm of fifty acres and twenty head of stock, the greatest marginal benefit comes not from more detailed measurement of any one enterprise, but from bringing the existing records of all enterprises into relation with one another.

2.5 Review of Existing Systems

A number of solutions are available to a farmer seeking to computerise. Their relative characteristics are summarised in Table 2.1.

Table 2.1 Comparison of existing farm management solutions

Solution type	Strengths	Limitations for the target user
Commercial cloud farm software (subscription)	Comprehensive features; professionally maintained; regular updates; mobile applications.	Recurring subscription in foreign currency; requires continuous internet connectivity; data held off site; features often designed for large-scale temperate agriculture.
Generic spreadsheets (Excel, Google Sheets)	Familiar; flexible; already installed on many machines; no additional cost.	No data validation; no referential integrity; no concurrent multi-user editing; no roles or permissions; no audit trail; formulas easily broken by ordinary editing.
General accounting packages	Sound financial control; recognised reporting formats; supports statutory returns.	Financial dimension only; no concept of an animal, a field, a crop cycle or a store issue; cannot relate physical production to money.
Enterprise-specific software (e.g. herd management)	Deep functionality within one enterprise; specialised calculations.	Covers a single enterprise; a mixed farm would need several unconnected systems; typically licensed per unit.
Custom locally-hosted system (this project)	No recurring cost; operates without internet; covers all enterprises of a mixed farm; adaptable to local practice; data remains on the farm.	Requires initial development; maintenance depends on local skills; no vendor support arrangement.

Table 2.1 — Comparison of existing farm management solutions

The comparison exposes a clear vacancy. Systems that are affordable are not integrated, and systems that are integrated are neither affordable nor usable without connectivity. The design decision to build a locally hosted integrated system on free software is a direct response to that vacancy.

2.6 Review of Applicable Technologies

The technologies considered for the implementation, and the reasons for the choices made, are summarised in Table 2.2. In each case the decisive criteria were zero licence cost, capability of local operation without connectivity, and the availability of the skills required to maintain the system after the project ends.

Table 2.2 Summary of technologies reviewed

Layer	Options considered	Selected	Justification
Web server	Apache, Nginx, IIS	Apache (via XAMPP)	Bundled with XAMPP; runs on Windows, macOS and Linux; requires no configuration for this use.
Server language	PHP, Python (Django), Java (JSP), C# (ASP.NET)	PHP 8	Widely taught locally; no build or compilation step; deploys by copying files; large body of documentation.
Database	MySQL/MariaDB, PostgreSQL, SQLite, MS Access	MySQL / MariaDB	Relational integrity with foreign key enforcement; bundled with XAMPP; administered through phpMyAdmin.
Data access	mysqli, PDO, raw queries	PDO with prepared statements	Database-independent interface and parameterised queries, which eliminate

Layer	Options considered	Selected	Justification
			SQL injection by construction.
Interface	Bootstrap, Tailwind, hand-written CSS	Hand-written CSS	Frameworks are normally delivered from a content delivery network, which fails without internet access.
Charts and icons	Chart.js, Font Awesome, custom SVG	Purpose-built SVG	External chart and icon libraries are delivered from a content delivery network and fail without connectivity; inline SVG also inherits colour and can be animated.

Table 2.2 — Summary of technologies reviewed

2.7 Relational Database Theory and Normalisation

Normalisation is the procedure by which a relational schema is refined to eliminate redundancy and the update anomalies that redundancy produces. A relation is in first normal form when all its attributes are atomic; in second normal form when, in addition, every non-key attribute is fully functionally dependent on the whole of the primary key; and in third normal form when, further, no non-key attribute is transitively dependent on the primary key. The colloquial statement of third normal form — that every non-key attribute depends on the key, the whole key, and nothing but the key — captures the requirement adequately for practical design.

The database designed for this project is normalised to third normal form throughout. The practical consequences are visible in the schema. Livestock categories are held in a separate relation rather than repeated as text against every animal, so that renaming a category is a single-row update and misspelling a category name is impossible. Inventory movements are held separately from inventory items, so that the running balance can be derived and audited rather than merely asserted. Employees are separated from user accounts, because not every worker requires a login and not every login belongs to a field worker.

A deliberate and documented departure from strict normalisation occurs in the storage of the inventory item balance. The balance is logically derivable by summing the movement ledger, and holding it as a column is therefore a controlled redundancy. It is retained because the balance is read on almost every page of the inventory module while movements are appended comparatively rarely, and recomputing an aggregate on every read would degrade responsiveness. The redundancy is made safe by confining all balance changes to a single code path that writes the movement and updates the balance inside one database transaction, as described in Section 4.5.

2.8 Three-Tier Application Architecture

The three-tier architectural pattern separates an application into a presentation tier responsible for interaction with the user, an application tier containing business rules and processing, and a data tier responsible for persistent storage. Communication is permitted only between adjacent tiers.

The benefits of this separation are well established. Each tier may be modified without disturbing the others, so the interface can be redesigned without touching business rules and the database can be tuned without altering the interface. Responsibilities are localised, which shortens the path from a reported fault to the code responsible for it. And security controls can be concentrated at the boundary between the presentation and application tiers, which is the correct place for them, since the presentation tier executes on a machine the system does not control.

That last point deserves emphasis because it governs a decision taken repeatedly in this implementation. Anything enforced only in the browser — a hidden button, a disabled field, a JavaScript validation rule — is a convenience for the honest user and no obstacle whatever to a dishonest one, because the request that ultimately reaches the server can be composed by hand. Every rule that matters is therefore enforced again in the application tier, and Section 4.7 documents a test in which precisely such a forged request is submitted and correctly refused.

2.9 Security in Web Applications

The Open Web Application Security Project publishes a periodically revised list of the most critical web application security risks, which serves as the practical reference for defensive design. Four of those risks are directly relevant to a system of this kind.

Injection occurs when untrusted input is interpreted as part of a command or query; a query assembled by concatenating user input allows an attacker to alter its meaning. The accepted remedy is the prepared statement, in which the query structure is fixed before any value is supplied, so parameters can never be interpreted as syntax. Broken access control occurs when restrictions on what an authenticated user may do are not properly enforced, and arises characteristically from enforcing authorisation in the interface alone; the remedy is a server-side check on every operation that changes state.

Cross-site request forgery causes an authenticated user's browser to submit a request the user did not intend; the standard remedy is a secret token, bound to the session, which must accompany every state-changing request and which an attacker's page cannot read. Stored passwords deserve separate mention: they must never be recoverable, since the database may be compromised, and a modern adaptive hash such as bcrypt with a per-password salt and a deliberately high computational cost ensures that even full disclosure of the users table does not readily yield them. Section 4.6 documents how each measure was implemented.

2.10 Interface Design Principles

The usability literature offers well-established heuristics for interface design, of which several were adopted explicitly in this project. Visibility of system status requires that the system keep the user informed about what is happening; this is honoured through immediate confirmation messages after every operation. Recognition rather than recall requires that the user should not

have to remember information from one part of the interface to another; this motivates the consistent use of colour-coded status labels and icons throughout.

Consistency requires that users should not have to wonder whether different words or actions mean the same thing, so every module repeats an identical layout. Error prevention is preferred to good error messages, so destructive operations require a confirmation naming the specific record affected rather than a generic warning.

Progressive disclosure, finally, governs the arrangement of every page: summary indicators appear first, analytical charts second, and detailed record tables last, so that a manager seeking a quick assessment need not read a table of two hundred rows to obtain it.

2.11 Gap Analysis

First, integrated management information demonstrably improves farm decision making, but the systems that provide it are priced and provisioned for large commercial agriculture. Second, the tools that small farms can actually afford — paper and spreadsheets — do not provide integrity, concurrency, alerting or auditability. Third, almost all contemporary web-based solutions assume continuous connectivity, an assumption that does not hold at the farm locations considered here. Fourth, systems developed elsewhere embed practices and terminology that do not correspond to local farming reality.

The gap is therefore the absence of an integrated, multi-user, role-aware farm management system that runs on commodity local infrastructure, functions without an internet connection, carries no recurring cost, and reflects the enterprises and vocabulary of local mixed farming. It is that gap the present work is designed to fill.

2.12 Chapter Summary

This chapter established the informational nature of farm management and the specific characteristics of agriculture that make records valuable. It examined the documented deficiencies of manual record keeping, classified farm management information systems by functional coverage, and compared the solutions presently available to a small farmer, finding that affordability and integration are not currently available together. It reviewed the relational, architectural, security and usability principles applied in this project and justified each technology selected. Finally it identified the gap in existing provision that this project addresses. The following chapter analyses the requirements arising from this position and presents the design of the proposed system.

CHAPTER THREE
SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents the analysis of the existing system and the design of the proposed system. It begins with the development methodology adopted and the techniques used to gather facts about current practice. It then analyses the manual system in operation at the case study farm, states the functional and non-functional requirements derived from that analysis, and establishes the feasibility of the proposed solution. The remainder of the chapter presents the design itself: the system architecture, the use case model, the entity relationship model and data dictionary, the principal process flows, the data flow model, the access control design and the principles governing the user interface.

3.2 System Development Methodology

The iterative and incremental model was adopted for this project. Under this model the system is developed as a series of increments, each of which is taken through the full cycle of analysis, design, implementation and testing before the next is begun. Each increment produces a working, tested portion of the final system.

Three considerations governed this choice. First, the project operated to a fixed academic deadline, and the iterative model guarantees that a demonstrable system exists at every point, whereas the waterfall model concentrates all delivery risk at the end. Second, requirements for a system of this kind cannot be fully known in advance; showing farm staff a working livestock module elicited requirements for the health module that no interview had surfaced. Third, defects are found early. Two significant faults documented in Chapter Four were discovered during the testing of one increment and corrected before the modules that would have inherited them were built.

3.2.1 Fact-Finding Techniques

Three complementary techniques were employed to establish how the farm actually operates.

Interviews were conducted with the farm owner and the farm manager. Semi-structured in form, they began with the daily and seasonal cycle of work and moved to the decisions each person takes and the information each would wish to have when taking them. The interviews established the priority of management concerns and revealed, notably, that the question managers most wanted answered — profitability by individual crop — was not being answered at all under the existing arrangements.

Document analysis examined the paper registers in current use — the livestock notebook, purchase records, wage sheets and sales receipts — and proved the most productive source of data requirements, because the columns a farm has ruled into its own books are direct evidence of the attributes it considers significant. Several fields in the final schema, including the tag

number format and the recording of an acquisition cost against each animal, were taken directly from existing practice.

Observation of routine work established who records what and at what moment, revealing a constraint that shaped the production module: the person holding the information is frequently not the person who will later sit at the computer. The forms were therefore made completable in seconds with minimal typing, and the worker role was given permission to record production and health observations directly.

3.3 Analysis of the Existing System

The existing system is entirely manual. Records are entered by hand into separate exercise books held in the farm office: the livestock book lists animals with tag numbers and occasional treatment notes, a general notebook records purchases as they occur, wages are recorded on a monthly sheet, and sales on receipt counterfoils retained loose in a folder. Summarisation occurs at month end for wages and at season end for crops, performed by hand with a calculator. No summarisation of any kind is performed for the store, and no reconciliation is performed between purchases of feed and the production of the units consuming it.

3.3.1 Weaknesses of the Existing System

Analysis of current practice identified the following specific weaknesses, each of which generated a requirement for the proposed system:

1. Data duplication. The same purchase is written into more than one book, and the copies diverge when one is later corrected and the other is not.
2. Arithmetic error. Season-end totals are computed by hand across hundreds of entries, with no means of detecting a mistake once made.
3. Absence of validation. Nothing prevents a date being written in the wrong year, a tag number being duplicated, or a quantity being omitted.
4. No concurrent access. Only one person can consult a book at a time, and only in the office where it is kept.
5. No alerting. A reorder level exists only in the manager's memory, and a vaccination due date, once written, is not revisited.
6. No cross-enterprise analysis. Feed cost and egg revenue exist in different books and are never brought together.
7. Fragility. Books are subject to loss, water damage and insect damage, and no copy exists.
8. No accountability. Entries are unattributed, so a disputed stock issue cannot be resolved from the record.

3.3.2 The Proposed System

The proposed system replaces the separate books with a single relational database, accessed through a web interface by several users concurrently under role-based authorisation. It

validates every entry at the point of capture, computes all summaries continuously, derives alerts from live data, records every operation in an audit trail, and presents analytical output through dashboards and printable reports. Every weakness enumerated above is addressed by a specific feature of the design that follows.

3.4 System Requirements

3.4.1 Functional Requirements

The functional requirements state what the system must do. They were derived directly from the fact-finding described above and are enumerated in Table 3.1.

Table 3.1 Functional requirements

ID	Requirement	Priority
FR1	The system shall authenticate users by username or email address together with a password.	High
FR2	The system shall restrict access to functions according to the role held by the authenticated user.	High
FR3	The system shall permit creation, retrieval, modification and deletion of livestock records, including category, breed, sex, weight, age and status.	High
FR4	The system shall record veterinary health events against an animal, including the treatment date and any next-due date.	High
FR5	The system shall record daily production output by product and by source enterprise.	High
FR6	The system shall permit management of land parcels and of the crops planted upon them, tracking progress from planting to harvest.	High
FR7	The system shall record harvests with quantity, quality grade and revenue, and shall optionally post the revenue to the financial ledger.	High
FR8	The system shall maintain inventory items with reorder levels and shall record every stock movement in an auditable ledger.	High
FR9	The system shall prevent the issue of a quantity of stock greater than the balance held.	High
FR10	The system shall permit tasks to be created, assigned to employees, prioritised and tracked to completion.	Medium
FR11	The system shall record income and expenditure under user-defined categories with a payment method and reference.	High
FR13	The system shall present a dashboard of key performance indicators and analytical charts.	High
FR14	The system shall generate alerts for stock below reorder level, overdue tasks and due veterinary treatments.	High
FR16	The system shall record every operation performed in an audit log identifying the user, module, action and time.	High
FR17	The system shall permit an administrator to create, modify, suspend and delete user accounts and to reset passwords.	High

Table 3.1 — Functional requirements of the proposed system

3.4.2 Non-Functional Requirements

The non-functional requirements state the qualities the system must exhibit while performing its functions. They are enumerated in Table 3.2.

Table 3.2 Non-functional requirements

ID	Category	Requirement
NFR1	Performance	Any page shall render within two seconds on commodity hardware with the expected data volume.
NFR2	Security	Passwords shall never be stored in a form from which they can be recovered.
NFR3	Security	All database access shall use parameterised queries.
NFR4	Security	Authorisation shall be enforced on the server for every operation that changes state.
NFR5	Usability	A user familiar with one module shall be able to operate any other module without further training.
NFR6	Portability	The system shall operate on Windows, macOS and Linux without modification of the source.
NFR7	Availability	The system shall operate with no internet connection at run time.
NFR8	Compatibility	The interface shall be usable on desktop, tablet and mobile screen widths.
NFR9	Reliability	Stock balances shall remain consistent with the movement ledger under all conditions, including failure part-way through an update.
NFR10	Auditability	It shall be possible to determine which user performed any recorded operation and when.
NFR11	Maintainability	Presentation, business logic and data access shall be separated so that each may be modified independently.
NFR12	Accessibility	Colour shall not be the sole means of conveying information, and the interface shall respect a user preference for reduced motion.

Table 3.2 — Non-functional requirements of the proposed system

3.5 Feasibility Study

3.5.1 Technical Feasibility

The system is technically feasible. Every component required — the Apache web server, the MySQL database server, the PHP interpreter and a web browser — is contained in the XAMPP distribution, which is free, mature and installable on ordinary hardware, and the skills required are those taught within the programme.

3.5.2 Economic Feasibility

The system is economically feasible. All software employed is free of licence charge, and the system runs on a computer of a specification the farm already possesses. The only costs are the development effort, which is borne as academic work, and the operator training, which is minimal because the interface follows the structure of the paper registers already in use. Set against this, avoiding a single feed stock-out or a single missed vaccination in a season

represents a material saving; the return on the investment is therefore realised almost immediately.

3.5.3 Operational Feasibility

The system is operationally feasible. The data entry forms were deliberately modelled on the columns of the existing paper registers, so that staff enter the same information in the same order and in the same vocabulary they already use. The three-role structure mirrors the existing division of authority on the farm between owner, manager and worker, rather than imposing a new one. Resistance to adoption is therefore expected to be low.

3.6 System Architecture

The system is constructed on the three-tier architectural pattern described in Section 2.9. Figure 3.1 shows the arrangement of the tiers and the principal components of each.

The presentation tier executes in the browser and comprises the generated HTML, the stylesheet, the interface JavaScript with its charting engine, and the inline icon library. The application tier executes on the server under Apache and PHP, containing authentication, authorisation, module business logic, input validation, the security layer, audit logging and the reporting engine. The data tier comprises the MySQL database and the PDO abstraction through which it is reached, including transaction control and referential integrity. Communication is strictly between adjacent tiers, which is what allows every security control to sit at a single boundary the user cannot bypass.

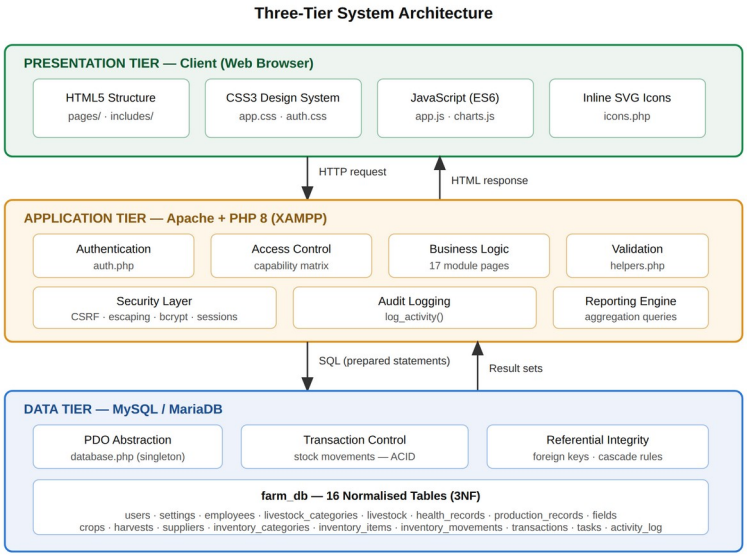

Figure 3.1 — Three-tier architecture of the Farm Management System

3.7 Use Case Modelling

The use case model identifies the actors who interact with the system and the operations each may perform. Three actors were identified from the fact-finding, corresponding to the three levels of authority already recognised on the farm.

The Administrator is the farm owner. This actor holds unrestricted access, including the creation and suspension of user accounts and the configuration of the system itself. The Farm Manager conducts the day-to-day running of the farm: managing livestock, crops, stores and staff, and recording financial transactions, but without authority over user accounts or system configuration. The Farm Worker records the work actually performed — production output, health observations, harvests and the status of assigned tasks — and may read but not alter the records of others.

Figure 3.2 presents the complete use case diagram.

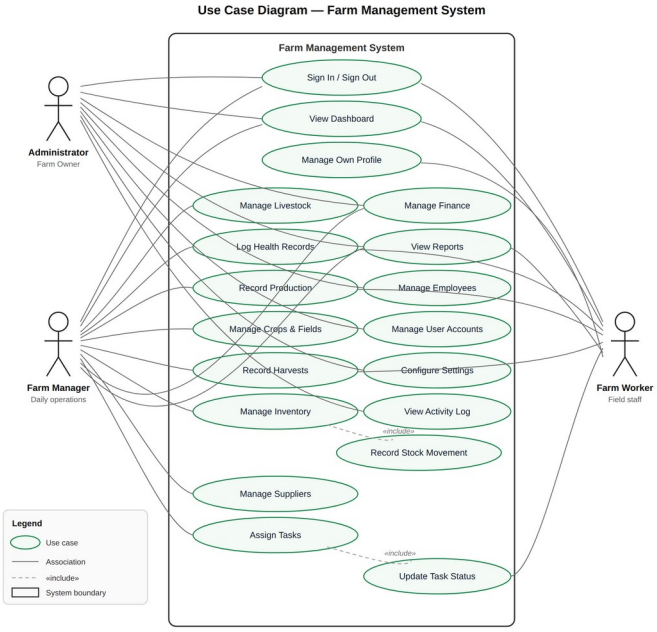

Figure 3.2 — Use case diagram showing the three actors and their permitted operations

Each use case was specified in narrative form during design, stating the actors, pre-conditions, main flow, alternative flows and post-condition; the stock movement case is realised in the algorithm of Figure 3.5.

3.8 Database Design

The database, named farm_db, comprises sixteen tables normalised to third normal form. Figure 3.3 presents the entity relationship diagram. Because the diagram is necessarily detailed, a full-page version is reproduced in Appendix A.

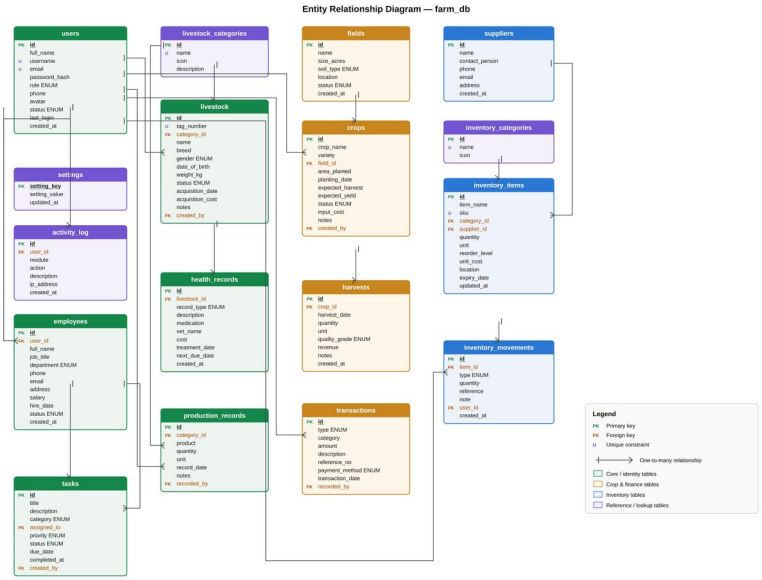

Figure 3.3 — Entity relationship diagram of the farm_db database

The design organises the tables into four groups. The identity group comprises users, employees, activity_log and settings, and establishes who may use the system and what they did. The livestock group comprises livestock_categories, livestock, health_records and production_records. The crop group comprises fields, crops and harvests, together with the transactions table which records the financial consequence of production. The inventory group comprises suppliers, inventory_categories, inventory_items and inventory_movements. The tasks table links the identity group to work performed across all enterprises.

Several design decisions in the schema merit explanation. Employees and users are held separately because the two populations overlap only partially: a casual worker has an employee record but no login, while an administrator may have a login and no employee record. A nullable foreign key joins them where both exist. Livestock categories are held as a table rather than an enumerated column so that a farm adopting a new enterprise, such as rabbits or fish, can add it without a schema change. And inventory movements are held as a ledger rather than the balance being edited directly, for the reasons of auditability set out in Section 2.8.

3.8.1 Data Dictionary

The full definition of every table is contained in the accompanying SQL schema file and reproduced in Appendix B. One representative table is documented here in full to establish the level of specification applied throughout; the remainder follow the same pattern and are given in Appendix B.

Table 3.3 Data dictionary — users table

Field	Data type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Surrogate primary key
full_name	VARCHAR(120)	NOT NULL	Name of the account holder
username	VARCHAR(60)	NOT NULL, UNIQUE	Sign-in identifier

Field	Data type	Constraint	Description
email	VARCHAR(140)	NOT NULL, UNIQUE	Alternative sign-in identifier
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hash of the password
role	ENUM('admin','manager','worker')	NOT NULL, DEFAULT worker	Authorisation level
phone	VARCHAR(30)	NULL	Contact telephone number
avatar	VARCHAR(180)	NULL	Optional profile image path
status	ENUM('active','suspended')	NOT NULL, DEFAULT active	Whether the account may sign in
last_login	DATETIME	NULL	Timestamp of the most recent sign-in
created_at	DATETIME	NOT NULL, DEFAULT NOW()	Record creation timestamp

Table 3.3 — Data dictionary for the users table

3.8.3 Referential Integrity Rules

The deletion rule attached to each foreign key was chosen deliberately according to whether the child record retains meaning once the parent has been removed. Where the child cannot exist independently, deletion cascades; where the child is a historical fact that survives its author, the reference is set to null. Table 3.4 records these decisions.

Table 3.4 Referential integrity rules

Relationship	Rule	Justification
livestock → health_records	CASCADE	A treatment record has no meaning once the animal it describes is removed.
crops → harvests	CASCADE	A harvest is an event belonging to a specific planting.
inventory_items → inventory_movements	CASCADE	A stock card describes the movements of one item only.
livestock_categories → livestock	CASCADE	Removing a category removes the enterprise it represents.
users → transactions	SET NULL	The financial record is a historical fact that must survive the departure of the user who entered it.
users → livestock, crops	SET NULL	The animal or crop continues to exist independently of the user who created the record.
users → activity_log	SET NULL	The audit entry must survive deletion of the account, or the audit trail could be erased by deleting a user.
employees → tasks	SET NULL	A task outlives the employment of the person assigned, and becomes unassigned.
fields → crops	CASCADE	A planting is defined by the parcel on which it stands.

Table 3.4 — Referential integrity rules and their justification

3.9 Process Design

Three processes were modelled as flowcharts because their logic determines the correctness or the security of the system. Figure 3.4 shows the authentication process, Figure 3.5 the stock movement algorithm, and Figure 3.6 the sequence of guards applied to every operation that writes to the database.

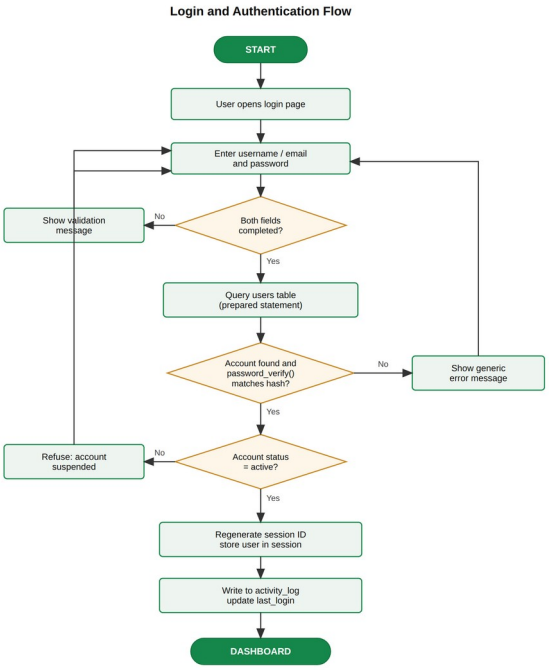

Figure 3.4 — Flowchart of the user authentication process

Figure 3.4 shows two decisions of consequence. A failure to find the account and a failure to verify the password converge on a single, identically worded message, since distinguishing them would allow an attacker to enumerate valid usernames; and the session identifier is regenerated immediately upon success, which defeats session fixation.

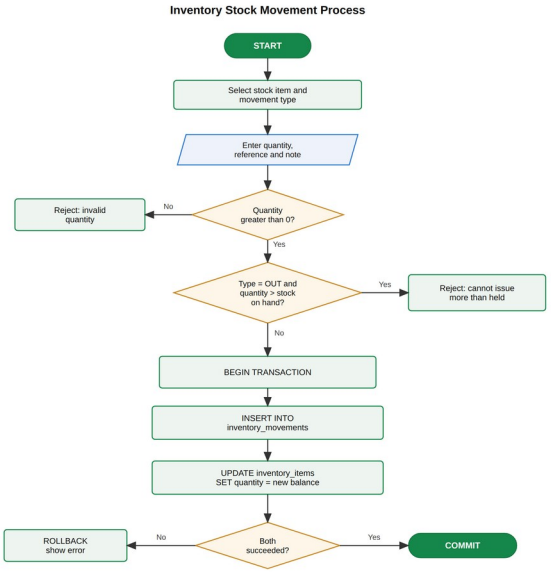

Figure 3.5 — Flowchart of the transactional stock movement algorithm

The stock movement flowchart in Figure 3.5 shows the two validations applied before any write occurs, and the transaction boundary enclosing the two writes that follow. Because the movement insert and the balance update are enclosed in a single transaction, a failure between them causes both to be discarded, and the ledger can never disagree with the balance.

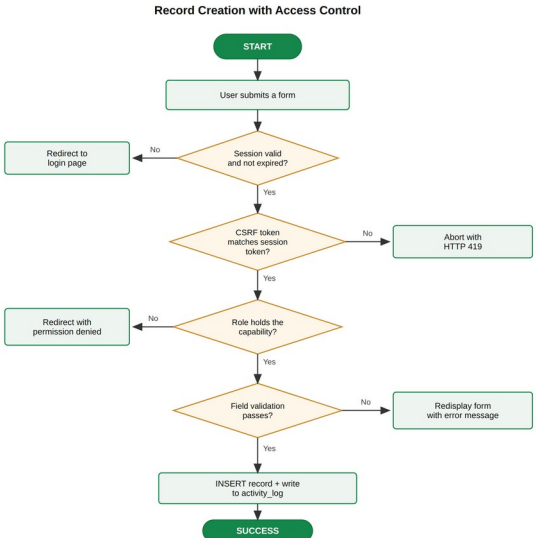

Figure 3.6 — Server-side guard sequence applied to every write operation

3.10 Data Flow Modelling

The data flow model describes the movement of data between external entities, processes and data stores. Figure 3.7 presents the context diagram, which treats the system as a single process and establishes its boundary with the outside world.

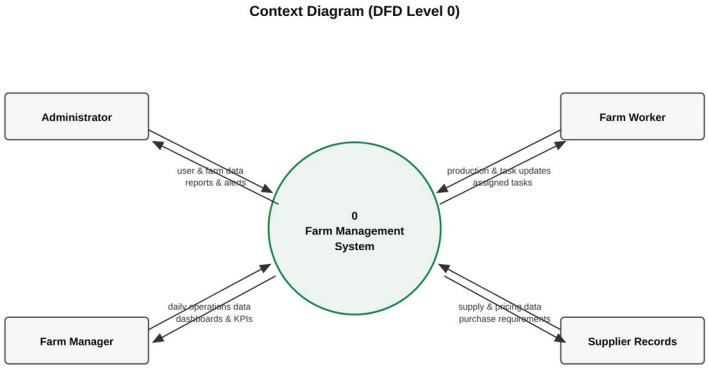

Figure 3.7 — Context level data flow diagram

Figure 3.8 decomposes the single process of the context diagram into the seven major processes of the system, showing the nine data stores each reads from and writes to. A full-page version appears in Appendix A.

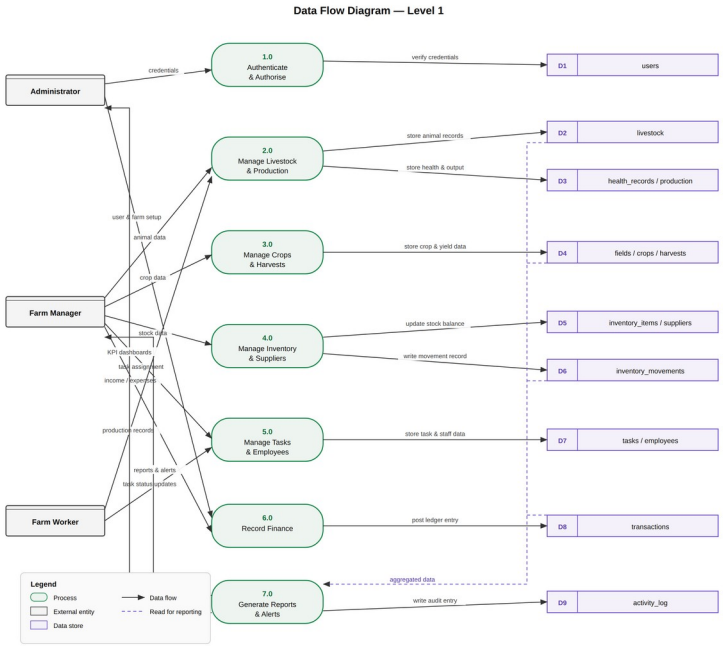

Figure 3.8 — Level 1 data flow diagram showing seven processes and nine data stores

3.11 Access Control Design

Authorisation is expressed as a capability matrix held in a single source file rather than as role checks scattered through the code. This decision was taken for two reasons: the complete authorisation policy can be read and audited in one place, and a change of policy requires an edit in one location rather than a search through seventeen modules.

Table 3.5 presents the policy. Each capability is granted to a role explicitly; a capability conferring management of a resource implies the capability to view it, but the converse does not hold.

Table 3.5 Role capability matrix

Capability	Administrator	Manager	Worker
View the dashboard	Yes	Yes	Yes
Add, edit and delete livestock	Yes	Yes	No
View livestock records	Yes	Yes	Yes
Log health records and production	Yes	Yes	Yes
Manage crops and fields	Yes	Yes	No
Record harvests	Yes	Yes	Yes
Manage inventory items	Yes	Yes	No
Record stock movements	Yes	Yes	No
Manage suppliers	Yes	Yes	No
Create and assign tasks	Yes	Yes	No
Update the status of a task	Yes	Yes	Yes
View and record financial transactions	Yes	Yes	No

Capability	Administrator	Manager	Worker
View employee records	Yes	Yes	No
Add and edit employees	Yes	No	No
View reports	Yes	Yes	Yes
Manage user accounts	Yes	No	No
View the activity log	Yes	No	No
Change farm settings	Yes	No	No

Table 3.5 — Role capability matrix

It bears repeating that this policy is enforced in the application tier. The interface additionally hides controls a user may not operate, but that is a usability measure and not a security measure. The distinction is tested explicitly in Chapter Four, where a request to create a livestock record is submitted directly to the server while authenticated as a worker, and is refused.

3.12 User Interface Design

Five principles, drawn from the usability literature reviewed in Section 2.11, governed the design of the interface.

Progressive disclosure determines the vertical arrangement of every page: summary indicators first, analytical charts second, the detailed table last, so that a quick assessment does not require reading twenty rows. Consistency determines that every module repeats an identical structure — page heading, summary indicators, filter toolbar, data table, pagination — so learning one module teaches the other sixteen. Feedback determines that every operation produces a visible confirmation naming what was done, and that destructive operations require a confirmation naming the specific record affected, "Remove CT-001 (Adwoa)?" rather than "Are you sure?". Recognition over recall determines that status is conveyed by a coloured, labelled badge and a distinct icon rather than a code the user must memorise.

Accessibility determines that colour is never the sole carrier of meaning, that every status badge also carries a text label, that the interface remains operable at a width of 360 pixels, and that a user whose system requests reduced motion receives no animation.

A light and a dark presentation are both provided, selectable by the user and remembered between sessions, since farm offices vary considerably in ambient light.

3.13 Chapter Summary

This chapter has presented the analysis and design of the system: the methodology and fact-finding techniques applied, the weaknesses of the manual system, the requirements derived, and the feasibility of the proposal. The design was expressed through a three-tier architecture, a use case model, an entity relationship model of sixteen normalised tables, process flowcharts, a two-level data flow model, a capability matrix and the principles governing the interface. The next chapter documents its implementation and testing.

CHAPTER FOUR
SYSTEM IMPLEMENTATION AND TESTING

4.1 Introduction

This chapter documents the construction of the system designed in Chapter Three and the programme of testing applied to it. It describes the development environment and the organisation of the source code, presents each implemented module with an accompanying screenshot, explains the principal algorithms and the security measures implemented, and reports the results of unit, integration, system and security testing. It closes with the defects discovered during testing, the corrections applied, and the deployment procedure.

4.2 Development Environment

The system was developed and tested in the environment described in Table 4.1. All components are freely available and none requires a licence fee or a subscription.

Table 4.1 Development environment

Component	Selection	Role in the project
Server platform	XAMPP	Bundles Apache, MySQL/MariaDB and PHP in a single installation
Web server	Apache 2.4	Serves the application over HTTP
Server language	PHP 8	Executes all business logic and generates the HTML delivered to the browser
Database server	MySQL / MariaDB	Persistent storage with referential integrity and transactions
Database access	PDO with prepared statements	Parameterised query interface preventing SQL injection
Database administration	phpMyAdmin	Schema inspection and manual import or export
Interface languages	HTML5, CSS3, JavaScript (ES6)	Structure, presentation and client-side behaviour
Code editor	Visual Studio Code	Source editing with syntax checking
Browsers used for testing	Chrome, Firefox and Edge	Cross-browser verification of layout and behaviour
Version control	Git	Change history and backup of the source code

Table 4.1 — Development environment

4.3 Organisation of the Source Code

The source code is organised so that the three tiers of the architecture correspond to distinct directories. Configuration and the database access layer are held in config/, shared services — authentication, the capability matrix, validation, escaping, audit logging and the icon library — in includes/, the design system and client scripts in assets/, and the seventeen functional modules in pages/. The database schema and demonstration data are held in database/farm_db.sql, and the supporting documentation in docs/. This arrangement means that

a change to the interface touches only assets/, and a change to a business rule touches only the module that owns it.

4.4 Implementation of System Modules

All seventeen modules were implemented. Six representative screens are shown here; the remainder are described in the text and reproduced at full page size in Appendix A. Every screenshot was taken from the working system operating against the populated database.

4.4.1 Authentication and Installation

The sign-in screen, shown in Figure 4.1, presents a split layout: an illustrated panel carrying live figures drawn from the database alongside a focused sign-in form. Credentials may be supplied as either a username or an email address, and three buttons fill in the credentials of each role so that the difference between them can be demonstrated without typing.

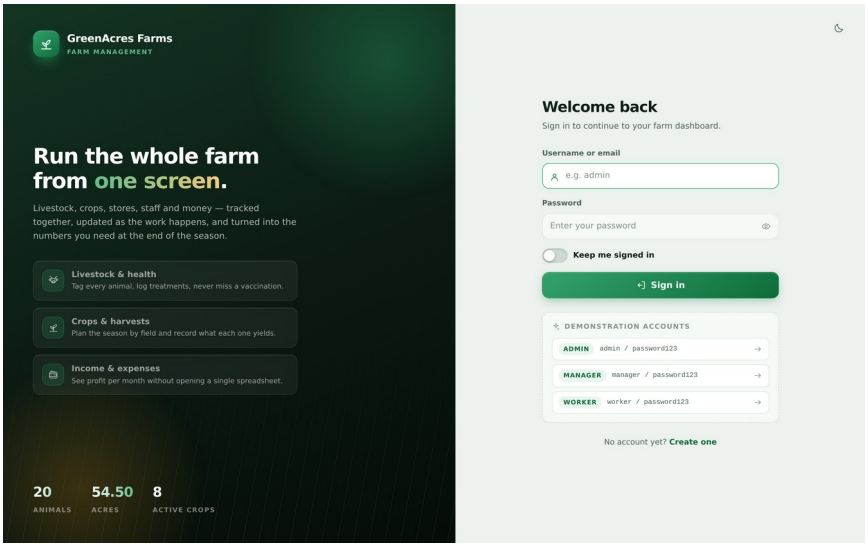

Figure 4.1 — The sign-in screen

The screen implements two of the security behaviours described in Section 3.9: a failed sign-in produces a single message that does not disclose whether the account or the password was at fault, and the session identifier is regenerated upon success.

To remove the need for manual database preparation, a web-based installer was also written. It verifies five environmental conditions — the PHP version, the presence of the PDO MySQL extension, the readability of the schema file, the writability of the uploads directory and the reachability of the database server — and only then offers to build the database. It creates the database, executes the schema, loads the demonstration data and reports the row count of each principal table, before warning the operator to delete the installer, which would otherwise allow the database to be rebuilt and the data lost.

4.4.2 Dashboard

The dashboard, shown in Figure 4.2, is the operational overview of the whole farm. Four headline indicators occupy the top of the page — total livestock, crops under cultivation,

income for the current month and net profit — each with the percentage change against the preceding month. Below them the page presents a twelve-month cash flow chart, the composition of the herd, production for the current week, the items of stock that have fallen below their reorder level, the tasks falling due, the progress of each growing crop, and a feed of recent system activity. The complete page is reproduced as Plate A4.

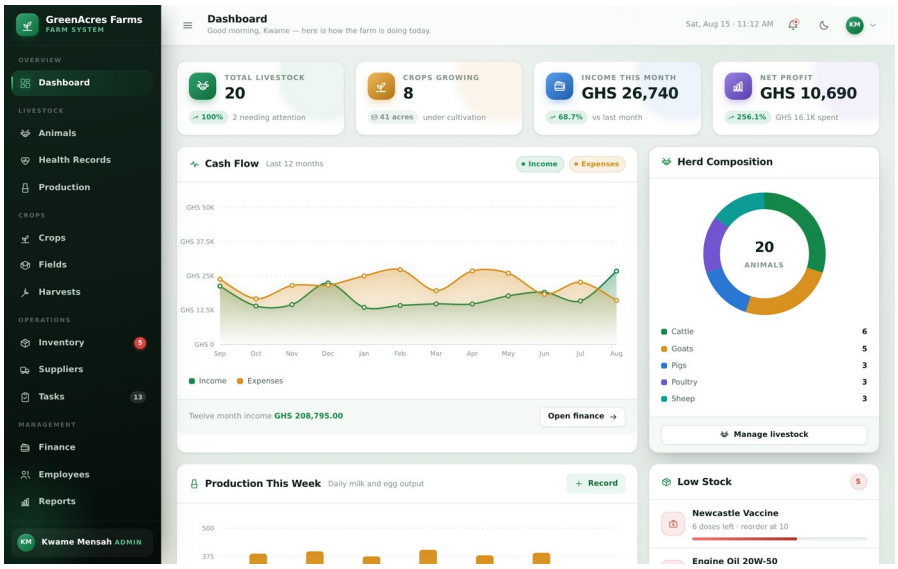

Figure 4.2 — The main dashboard

4.4.3 Livestock, Health and Production

The livestock register lists every animal with its tag number, category, breed, sex, derived age, weight, health status and acquisition value, and may be searched by tag, name or breed and filtered by category and status. Records are created and modified through a dialogue divided into an identification section and a physical and acquisition section, which validates that the tag number is unique before accepting the record.

The health module records vaccinations, treatments, check-ups, deworming and surgical procedures against individual animals. Each record carries a treatment date and an optional next-due date, and it is the latter that drives the reminders described in Section 4.5.4; records past their due date are marked in red and those falling due within fourteen days in amber. The production module records daily output such as milk and eggs, and was deliberately designed for rapid entry since the person recording production is usually doing so between other tasks. A fourteen-day trend chart plots each product separately, so that a decline in yield is seen immediately rather than inferred at the end of a month.

4.4.4 Crops, Fields and Harvests

The crop module records what has been planted, where, when and with what expectation of yield, displaying for each crop a progress bar computed from the elapsed proportion of the interval between planting and expected harvest. A comparison chart sets expected yield against the quantity actually harvested, which over successive seasons provides the evidence for revising yield expectations. The fields module records the land itself — the size of each parcel,

its soil type, its location and its current state — with a utilisation panel showing what proportion of each parcel is presently planted, so that idle land is visible at a glance.

The harvest module records what actually came off each field: quantity, unit, quality grade and revenue, computing the implied unit price so that the price realised can be compared across buyers and seasons. It offers, at the point of recording a harvest, to post the revenue directly into the financial ledger. This is the one place in the system where a single user action writes to two modules, and it exists because the alternative — requiring the same sale to be entered twice — is precisely the duplication the system was built to eliminate.

4.4.5 Inventory and Suppliers

The inventory module, shown in Figure 4.3, manages the farm store: feed, seed, fertiliser, agrochemicals, veterinary supplies, tools and fuel. Each item carries a reorder level, and items at or below that level are highlighted and raised as alerts.

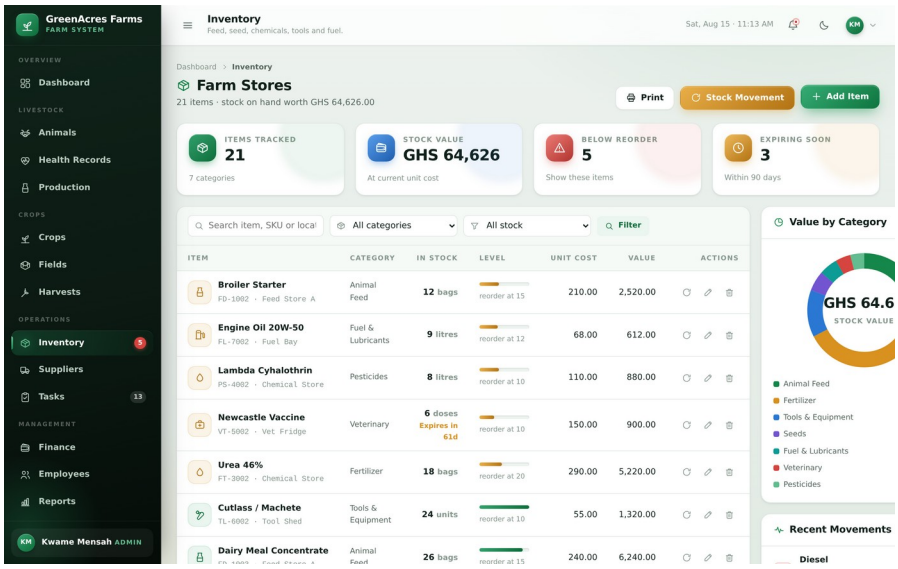

Figure 4.3 — Inventory management module

Stock balances are never edited directly. All changes pass through a movement dialogue distinguishing a receipt, an issue and a physical-count adjustment. The algorithm behind it is described in Section 4.5.2; its two important properties are that an issue exceeding the balance held is refused, and that the movement record and the balance update occur within a single database transaction. The supplier directory records the businesses from which inputs are purchased, showing for each the number of stock items sourced from them and their present value.

4.4.6 Tasks and Employees

The task module presents farm work in two forms. The board view arranges tasks into four columns by status, giving an immediate visual sense of the work in hand. A list view presents the same data in tabular form for printing and for review of a longer period.

Task status may be updated by any authenticated user, including a worker, while creation and assignment are reserved to managers and administrators. This division reflects the reality that the worker knows when a job is finished but does not decide what work is scheduled. The employee register records the workforce with job title, department, contact details, salary, hire date and employment status, computing the monthly payroll total and the average length of service.

4.4.7 Finance and Reporting

The finance module, shown in Figure 4.4, maintains the ledger of income and expenditure, each transaction carrying a type, a category, an amount, a payment method, a reference and a date. It presents four analytical views: a twelve-month comparison of income against expenditure, the composition of expenditure by category, the trend of net profit month by month, and a ranking of income sources. The complete page is reproduced as Plate A5.

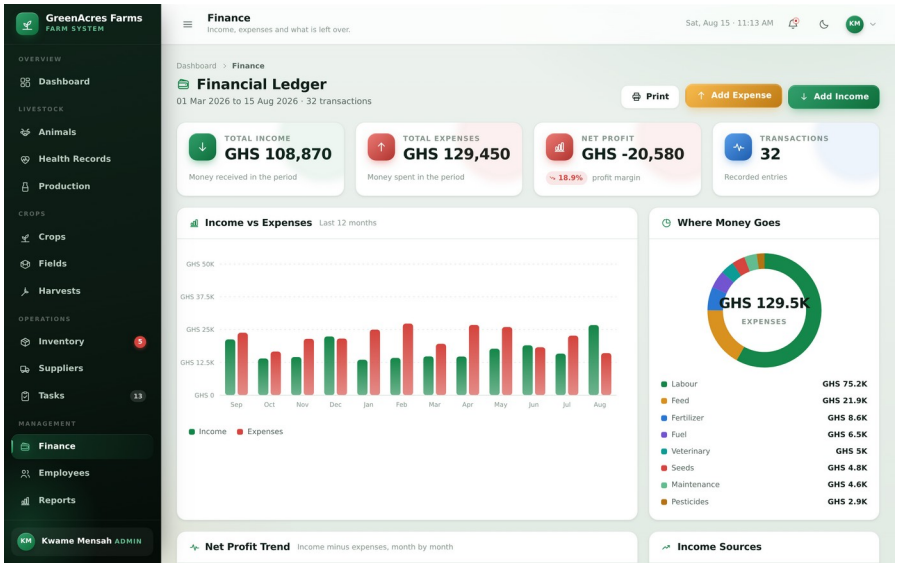

Figure 4.4 — Financial ledger

The reporting module consolidates the analytical output of every other module into a single document for a user-selected period, covering financial performance, key indicators across all modules, herd structure and health rates, crop performance including yield per acre and gross margin by crop, and a summary of livestock production. It is reproduced as Plate A6. The page carries a dedicated print stylesheet: when printed, the navigation, toolbars and buttons are suppressed and a letterhead carrying the farm name, location and reporting period is revealed, so that the output is a document suitable for presentation to a bank or a cooperative rather than a screen capture.

4.4.8 Administration, Auditing and Configuration

The user administration module is restricted to administrators. It permits accounts to be created, edited, suspended, reactivated and deleted, and passwords to be reset. Three safeguards are enforced: an administrator may not delete the account with which they are signed in, may not remove their own administrator rights, and may not delete the last remaining administrator

account. Without the third safeguard the system could be rendered permanently unadministrable by a single deletion. The page also displays the capability matrix of Table 3.6, so that the effect of assigning a role is visible at the moment of assignment.

The activity log is the audit trail. Every operation that changes data writes an entry recording the user, the module, the action, a description, the originating network address and the time. It may be filtered by module, by user and by action type, and entries older than a chosen age may be cleared by an administrator — an operation which is itself recorded in the log.

The settings module holds the farm profile that appears on printed reports, the currency and date formatting preferences, and the reference data through which the system is adapted to a particular farm. A category still referenced by existing records cannot be deleted, which prevents orphaned references. The profile module allows any authenticated user to maintain their own contact details and change their own password; a password change requires the current password, so that an unattended signed-in session cannot be used to lock out the legitimate account holder.

4.5 Implementation of Key Algorithms

4.5.1 Database Access Layer

All database access passes through a small set of helper functions built on PHP Data Objects. Every query is prepared and its values bound as parameters, so that a value can never be interpreted as SQL syntax. The following extract shows the core of the layer.

```
function q(string $sql, array $params = []): PDOStatement
{
    $stmt = db()->prepare($sql);
    $stmt->execute($params);
    return $stmt;
}

function insert(string $table, array $data): int
{
    $cols         = array_keys($data);
    $placeholders = array_map(fn($c) => ':' . $c, $cols);

    $sql = sprintf(
        'INSERT INTO `%s` (`%s`) VALUES (%s)',
        $table,
        implode('`, `', $cols),
        implode(', ', $placeholders)
    );

    q($sql, $data);
    return (int) db()->lastInsertId();
}
```

Because every module builds its queries through these helpers, the protection against SQL injection is structural rather than a matter of discipline: it is not possible to write an ordinary query in this system that concatenates user input into SQL.

4.5.2 Transactional Stock Movement

The stock movement algorithm implements the design of Figure 3.5. The new balance is computed according to the movement type, the two writes are enclosed within a transaction, and a failure of either causes both to be discarded.

```
$newQty = match ($type) {
    'in'         => (float) $item['quantity'] + $quantity,
    'out'        => (float) $item['quantity'] - $quantity,
    'adjustment' => $quantity,   // sets the physically counted figure
};

try {
    db()->beginTransaction();

    insert('inventory_movements', [
        'item_id'   => $itemId,
        'type'      => $type,
        'quantity'  => $quantity,
        'reference' => post_or_null('reference'),
        'user_id'   => current_user()['id'],
    ]);

    update('inventory_items', ['quantity' => max(0, $newQty)], $itemId);

    db()->commit();
} catch (Throwable $ex) {
    db()->rollBack();
    flash('danger', 'The stock movement could not be saved.');
}
```

The guard preventing an over-issue is applied before the transaction is opened, so that an invalid request never reaches the database at all.

4.5.3 Capability Checking

Authorisation is decided by a single function, can(), which consults the matrix of Table 3.6. Administrators hold a wildcard entry; managers and workers hold explicit lists. The rule that management of a resource implies the ability to view it is expressed once inside that function rather than repeated at every point of use, so a change of policy is a change in one place. Every page guard and every write handler calls it.

4.5.4 Derived Alerting

Alerts are derived from live data at the moment a page is requested rather than being stored when a condition first arises. Low stock is the condition quantity <= reorder_level; an overdue task is an open task whose due date has passed; a due treatment is a health record whose next-due date falls within the coming week. The consequence is that an alert can never be stale: an item restocked ceases to be reported as low the moment the movement is recorded, with no separate step to clear the alert.

4.5.5 The Charting Engine

Because the system must operate without an internet connection, the common practice of loading a charting library from a content delivery network was not available. A charting engine was therefore written for the project. It generates Scalable Vector Graphics directly in the browser and supports area, line, grouped bar, stacked bar, donut and sparkline forms, with shared tooltips and entry animations. Smooth curves are produced by converting the series points to cubic Bezier segments using a Catmull-Rom formulation, in which the control points of each segment are derived from the neighbouring points of the series, and axis maxima are rounded upward to a visually sensible interval so that gridline labels fall on round numbers.

4.6 Implementation of Security Measures

The security measures identified in Section 2.10 were implemented as described in Table 4.2.

Table 4.2 Security threats and countermeasures

Threat	Countermeasure implemented	Location in the code
SQL injection	All queries prepared with bound parameters; no query concatenates user input	config/database.php
Cross-site scripting	All dynamic output passed through an escaping function applying HTML entity encoding	includes/helpers.php
Cross-site request forgery	Per-session random token required on every state-changing request, compared with a timing-safe function	csrf_verify()
Password disclosure	Passwords stored as bcrypt hashes with per-password salt; never recoverable	includes/auth.php
Session fixation	Session identifier regenerated immediately upon successful authentication	attempt_login()
Session hijacking of an idle session	Automatic sign-out after a configurable period of inactivity	require_login()
Privilege escalation	Capability checked on the server in the handler of every write operation	require_capability()
User enumeration	Identical message returned whether the account or the password was wrong	attempt_login()
Unauthorised file upload	Uploads restricted by verified MIME type and by size; stored outside the executable path	handle_upload()
Loss of administrative access	The final administrator account cannot be deleted, demoted or suspended	pages/users.php

Table 4.2 — Security threats and the countermeasures implemented

The token comparison deserves a note. The verification uses a timing-safe comparison function rather than an ordinary equality test, because an ordinary comparison returns as soon as two bytes differ and the time it takes therefore leaks information about how much of the token was correct.

4.7 System Testing

4.7.1 Testing Strategy

Four levels of testing were applied. Unit testing examined individual functions in isolation, principally the formatting, validation and calculation helpers. Integration testing examined each module against the live database. System testing exercised complete workflows across modules, such as recording a harvest and confirming its appearance in the financial ledger. Security testing deliberately attempted to defeat the access control and request validation. All testing was conducted in the browser against the running system rather than by inspection of the code, so every result below reflects the behaviour of the assembled system as a user would encounter it.

4.7.2 Unit Test Results

Table 4.3 Unit test results

Function	Input	Expected output	Result
money()	12875	GHS 12,875.00	Pass
money_short()	128750	GHS 128.8K	Pass
percent_change()	(120, 100)	20.0	Pass
percent_change()	(50, 0)	100.0 (no division by zero)	Pass
days_until()	a date three days ahead	3	Pass
days_until()	a date two days past	−2	Pass
age_from()	a date of birth 2 years 4 months past	2y 4m	Pass
initials()	'Boatemaa Kyerewaa Jantuah'	BJ	Pass
validate()	required field left empty	error message returned	Pass
validate()	malformed email address	error message returned	Pass
is_unique()	an existing tag number	false	Pass
e()	a string containing <script>	entities encoded, no markup emitted	Pass

Table 4.3 — Unit test results

4.7.3 System and Security Test Results

Twenty-four test cases were specified and executed against the running system. The results are given in Table 4.4.

Table 4.4 System and security test cases

ID	Test case	Expected result	Actual	Verdict
TC-01	Sign in with valid credentials	Session created, redirected to dashboard	As expected	Pass
TC-02	Sign in with an incorrect password	Generic error, no session created	As expected	Pass
TC-03	Sign in to a suspended account	Access refused with explanation	As expected	Pass
TC-04	Create a livestock record	Record persisted and listed in the register	As expected	Pass

ID	Test case	Expected result	Actual	Verdict
TC-05	Open the edit dialogue for a record	Form pre-filled with the current values	As expected	Pass
TC-06	Modify a livestock record	Change persisted and reflected in the list	As expected	Pass
TC-07	Create an animal with a duplicate tag	Rejected with a validation message	As expected	Pass
TC-08	Delete a livestock record	Animal and its health records removed	As expected	Pass
TC-09	Submit a form with a required field empty	Submission blocked, field highlighted	As expected	Pass
TC-10	Record a stock receipt of 10 units	Balance increases by exactly 10	As expected	Pass
TC-11	Issue more stock than is held	Rejected, balance unchanged	As expected	Pass
TC-12	Inspect the movement ledger after a movement	Movement recorded with user and reference	As expected	Pass
TC-13	Change the status of a task	Status and completion time updated	As expected	Pass
TC-14	Record a financial transaction	Ledger and period totals updated	As expected	Pass
TC-15	Submit a request with an invalid CSRF token	Request refused with HTTP 419	As expected	Pass
TC-16	Worker opens the finance page	Redirected with a permission message	As expected	Pass
TC-17	Worker submits a forged create request to the server	Refused server-side; no record created	As expected	Pass
TC-18	Manager opens user administration	Redirected with a permission message	As expected	Pass
TC-19	Request a protected page without signing in	Redirected to the sign-in page	As expected	Pass
TC-20	Leave a session idle beyond the timeout	Signed out with an explanatory message	As expected	Pass
TC-21	Compare dashboard totals with hand calculation	Figures agree	As expected	Pass
TC-22	Load every page with the network disconnected	All charts render, no console errors	As expected	Pass
TC-23	View the interface at 360 pixels width	Layout usable, no horizontal page scroll	As expected	Pass
TC-24	Print the report page	Navigation suppressed, letterhead shown	As expected	Pass

Table 4.4 — System and security test cases and their results

All twenty-four test cases passed. Test case TC-17 is the most significant of the security tests and warrants explanation. Signed in as a worker — a role for which the interface displays no control for creating an animal — a request to create a livestock record was composed and submitted directly to the server, carrying a valid session cookie and a valid cross-site request forgery token harvested from a page the worker is permitted to view. The request was refused by the capability check in the request handler and no record was created. This confirms that authorisation is enforced in the application tier and does not depend on the absence of a button in the interface.

4.7.4 Defects Identified and Resolved

Testing found two defects of substance, both in code shared across modules and therefore affecting the system widely. Both were traced, corrected and re-tested. They are reported in

Table 4.5 because the discovery and correction of genuine faults is part of the evidence that the testing was effective.

Table 4.5 Defects identified during testing

ID	Defect	Cause	Resolution
DF-01	On desktop displays the entire application appeared shifted one column to the right, with page content compressed into a narrow strip.	The mobile navigation backdrop element was given its positioning rules only inside a mobile media query. At desktop widths it therefore remained a normal element and was treated as a column of the page grid.	The element was given fixed positioning in the base stylesheet so that it is removed from the layout flow at every screen width, with only its visibility varying by breakpoint.
DF-02	The inventory page raised undefined-index warnings and then failed with a fatal error, while every other module worked correctly.	The shared layout file used the variable name $items for its navigation loop. Because the layout is included after the page has run its queries, the loop overwrote the inventory page's own $items result set with navigation data.	All variables internal to the layout file were renamed with a distinguishing prefix, eliminating the collision and preventing its recurrence in future modules.

Table 4.5 — Defects identified during testing and their resolution

Both defects illustrate a general lesson. Neither was a mistake in the logic of any single module; both arose at the boundary between shared code and the modules that consume it, and neither would have been found by inspecting a module in isolation. This is a direct argument for integration testing of the assembled system, which is how both were in fact discovered.

4.8 System Deployment

Deployment requires four steps: the project directory is copied into the web root of the XAMPP installation, Apache and MySQL are started from the control panel, the installation wizard is opened in a browser and executed, and the wizard file is then deleted. For multi-user operation over a local area network, other machines reach the system by substituting the server machine's network address for "localhost", provided the firewall permits inbound connections on the web server port; no installation is required on client machines, which need only a browser.

Two configuration steps are recommended before the system is used in earnest. The debug flag in the configuration file should be set to false, so that no diagnostic message can be displayed to a user; and the demonstration passwords should be changed. Both are documented in the setup guide reproduced in Appendix D.

4.9 Chapter Summary

This chapter documented the implementation and testing of the system: the development environment, the seventeen modules, the algorithms governing database access, transactional stock movement, capability checking and alerting, and the security measures applied against each identified threat. Twenty-four system tests and twelve unit tests all passed, including one confirming that authorisation cannot be circumvented by a request submitted directly to the server, and two genuine defects were found and corrected.

CHAPTER FIVE
SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter closes the report. It summarises the work undertaken, assesses the extent to which each objective stated in Chapter One was achieved, states the conclusions drawn from the study, and offers recommendations both to the case study farm and to those who may extend the work in future.

5.2 Summary of the Study

The study set out to address the failure of manual, paper-based record keeping to convert the data a farm already produces into information the farm can act upon. Chapter One established the background to the problem at GreenAcres Farms, a mixed farming enterprise at Ejisu in the Ashanti Region, and stated the aim, objectives and scope of the work.

Chapter Two reviewed the literature on farm management, agricultural record keeping and farm management information systems, together with the relational, architectural, security and usability principles applicable to such a system, and identified the gap: no affordable, integrated, offline-capable system exists for the local mixed farm. Chapter Three presented the analysis and design — eight weaknesses of the manual system, eighteen functional and twelve non-functional requirements, and a design expressed through a three-tier architecture, a use case model, an entity relationship model of sixteen normalised tables, process flowcharts, a two-level data flow model and a capability matrix. Chapter Four documented the implementation of seventeen modules and the testing applied to them, in which twelve unit tests and twenty-four system and security test cases all passed and two genuine defects were found and corrected.

The delivered system comprises approximately fifteen thousand lines of source code across thirty-nine files, sixteen database tables, seventeen functional modules and a complete set of supporting documentation.

5.3 Achievement of Objectives

Table 5.1 assesses each objective stated in Section 1.5 against the delivered system.

Table 5.1 Achievement of project objectives

#	Objective	How it was achieved	Status
1	Design a normalised relational database covering every farm enterprise	Sixteen tables in third normal form, with foreign keys and deliberate deletion rules; documented in Section 3.8 and Appendix B	Achieved
2	Implement secure multi-user access with role-based authorisation	Three roles governed by a single capability matrix, enforced in the application tier and verified by test cases TC-16 to TC-18	Achieved
3	Develop complete record management for all farm functions	Seventeen modules providing full create, read, update and delete operations, presented in Section 4.4	Achieved

#	Objective	How it was achieved	Status
4	Generate automatic operational alerts	Alerts for low stock, overdue tasks and due treatments, derived from live data so that they cannot become stale	Achieved
5	Produce analytical dashboards and printable reports	A dashboard of live indicators and charts, and a dedicated reporting module with a print stylesheet	Achieved
6	Maintain a complete audit trail	Every state-changing operation writes to the activity log with user, module, action, address and time	Achieved
7	Test for correctness, integrity and security	Twelve unit tests and twenty-four system and security tests executed, all passing; two defects found and corrected	Achieved

Table 5.1 — Achievement of project objectives

5.4 Findings and Discussion

Four findings emerged from the work which are worth stating separately from the objectives.

The first concerns the value of integration over depth. The fact-finding revealed that the questions farm managers most wanted answered were not questions about any single enterprise, but questions that crossed enterprise boundaries: whether the feed bought for the poultry unit was justified by the eggs it produced, or which crop returned the most per acre. Such questions cannot be answered by a deeper record of one enterprise; they require only a modest record of all of them, held in relation. This finding justified the decision to build an integrated system of moderate depth rather than a detailed system for one enterprise.

The second concerns the discipline imposed by the offline constraint. Building without an internet connection removed the option of loading a charting library, an icon font or a styling framework from a content delivery network. Meeting that constraint required writing a charting engine and an icon library from first principles. The result is a system with no runtime dependency of any kind, which cannot be broken by a remote service changing or disappearing, and every line of which can be explained during an examination. A constraint initially regarded as a limitation therefore produced a better engineering outcome.

The third concerns the location of security controls. Test case TC-17 demonstrated concretely what the literature asserts in principle: that hiding a control in the interface provides no security whatever, since the request that reaches the server can be composed by hand. Only the server-side capability check refused the forged request. The practical lesson, applied uniformly throughout this implementation, is that the interface may be designed for usability but authorisation must be decided again on the server.

The fourth concerns the character of the defects found. Neither substantive defect lay in the logic of an individual module; both arose at the boundary between shared code and the modules consuming it, and neither would have been detected by examining a module alone. This is direct evidence for the value of testing the assembled system rather than its parts in isolation.

5.5 Conclusion

The study set out to determine whether a genuinely useful farm management information system could be built for a small mixed farm on freely available, locally hosted technology, without recurring cost and without dependence on an internet connection. The evidence presented in this report supports the conclusion that it can.

The delivered system consolidates records previously scattered across several notebooks into a single relational database of sixteen normalised tables. It enforces the division of authority that already exists on the farm, and enforces it where enforcement is effective. It converts records into information continuously rather than at the end of a season, so that the profitability of a crop, the value of the store and the health status of the herd are available at the moment they are wanted rather than after an afternoon of arithmetic. It surfaces the conditions requiring action — stock below its reorder level, a treatment falling due, a task passing its deadline — before rather than after the loss has been incurred. And it records who did what, so that a disputed entry can be resolved from the record.

Each of the eight weaknesses identified in the existing manual system in Section 3.3.1 is addressed by a specific and tested feature of the delivered system. The objectives stated in Chapter One have been achieved in full, and the testing reported in Chapter Four supports the claim that the system behaves correctly and resists the common classes of web application attack.

More broadly, the work demonstrates that the barrier to computerised farm management at this scale is not technical. The technology required is free, mature and installable on hardware such farms already possess. The barrier has been the absence of a system shaped to the enterprises, the vocabulary and the operating conditions of the local mixed farm. This project supplies one such system, together with the documentation required for it to be understood, maintained and extended by others.

5.6 Recommendations

5.6.1 Recommendations to the Farm

The following are recommended to the management of the case study farm and to any farm adopting the system.

1. Enter the opening position completely before relying on the system. Livestock, land, stock balances and staff should all be recorded at the outset, since partial data produces misleading summaries and undermines confidence in the system.
2. Record production and stock movements daily rather than in weekly batches. The value of the alerting depends entirely on the currency of the underlying data.
3. Set realistic reorder levels for every stock item, and review them after one full season against actual consumption.

4. Give each worker their own account rather than sharing one. The audit trail is only meaningful if entries are attributable to individuals.
5. Change the demonstration passwords immediately, and set the debug flag to false before the system carries real data.
6. Export a database backup weekly through phpMyAdmin and keep the copy on separate media, away from the machine running the system.

5.6.2 Recommendations for Future Development

The following extensions were identified during the work as being of value but outside the scope agreed for this project.

1. Short message service alerting. Delivering low-stock and treatment-due alerts by text message through a local gateway would remove the requirement that a user open the system in order to see them.
2. A mobile capture application. A small application for capture at the point of work — in the milking parlour or at the store door — would shorten the interval between an event and its recording, and would remove the transcription step that currently stands between them.
3. Barcode or radio-frequency animal identification. Scanning a tag rather than typing it would speed data entry and eliminate transcription errors in the single field on which the livestock records depend.
4. Multi-farm tenancy. Extending the schema so that one installation can serve several independent farms would allow a cooperative to host the system on behalf of its members.
5. Predictive analytics. Once several seasons of harvest data have accumulated, yield forecasting from historical performance, soil type and planting date becomes feasible and would materially assist season planning.
6. Statutory accounting integration. Exporting the financial ledger in a form accepted by standard accounting packages would connect the management accounts maintained here to formal financial reporting.

5.6.3 Recommendations to the Institution

It is recommended that projects of this kind be encouraged to test against the operating conditions of the intended user rather than those of the laboratory. The single most consequential design decision in this project — building without any dependency on external network services — followed directly from taking seriously the fact that the intended user has intermittent connectivity. Requiring that constraint to be stated and tested would improve the practical value of student systems generally.

It is further recommended that project assessment give explicit credit for documented defect discovery. The two faults reported in Section 4.7.4 are evidence that the testing performed was genuine rather than nominal, and a convention that rewards their disclosure rather than encouraging their concealment would improve the instructional value of project reports.

REFERENCES

Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377–387.

Connolly, T. and Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation and Management. 6th edn. Harlow: Pearson Education.

Elmasri, R. and Navathe, S. B. (2016). Fundamentals of Database Systems. 7th edn. Boston: Pearson.

Fowler, M. (2003). Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.

Kay, R. D., Edwards, W. M. and Duffy, P. A. (2019). Farm Management. 9th edn. New York: McGraw-Hill Education.

Nielsen, J. (1994). Enhancing the Explanatory Power of Usability Heuristics. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 152–158.

Nielsen, J. and Molich, R. (1990). Heuristic Evaluation of User Interfaces. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 249–256.

Open Web Application Security Project (2021). OWASP Top 10: The Ten Most Critical Web Application Security Risks. Available at: https://owasp.org/Top10/ (Accessed: August 2026).

Oracle Corporation (2024). MySQL 8.0 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.0/en/ (Accessed: August 2026).

Pressman, R. S. and Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach. 9th edn. New York: McGraw-Hill Education.

Provost, F. and Fawcett, T. (2013). Data Science for Business. Sebastopol: O'Reilly Media.

Sommerville, I. (2016). Software Engineering. 10th edn. Harlow: Pearson Education.

Sørensen, C. G., Fountas, S., Nash, E., Pesonen, L., Bochtis, D., Pedersen, S. M., Basso, B. and Blackmore, S. B. (2010). Conceptual Model of a Future Farm Management Information System. Computers and Electronics in Agriculture, 72(1), 37–47.

The PHP Group (2024). PHP Manual: PHP Data Objects. Available at: https://www.php.net/manual/en/book.pdo.php (Accessed: August 2026).

The PHP Group (2024). PHP Manual: Password Hashing Functions. Available at: https://www.php.net/manual/en/book.password.php (Accessed: August 2026).

Tsiligiridis, T. A. and Ainali, K. (2018). Farm Management Information Systems: A Review. Journal of Agricultural Informatics, 9(2), 1–15.

World Wide Web Consortium (2023). Web Content Accessibility Guidelines (WCAG) 2.2. Available at: https://www.w3.org/TR/WCAG22/ (Accessed: August 2026).

APPENDIX A — FULL-PAGE DIAGRAMS AND SCREENSHOTS

The plates below reproduce the wider design diagrams and the principal interface screens at full page size, in landscape orientation, so that detail compressed in the body of the report remains legible in print. All screenshots were captured from the working system running against the populated demonstration database.

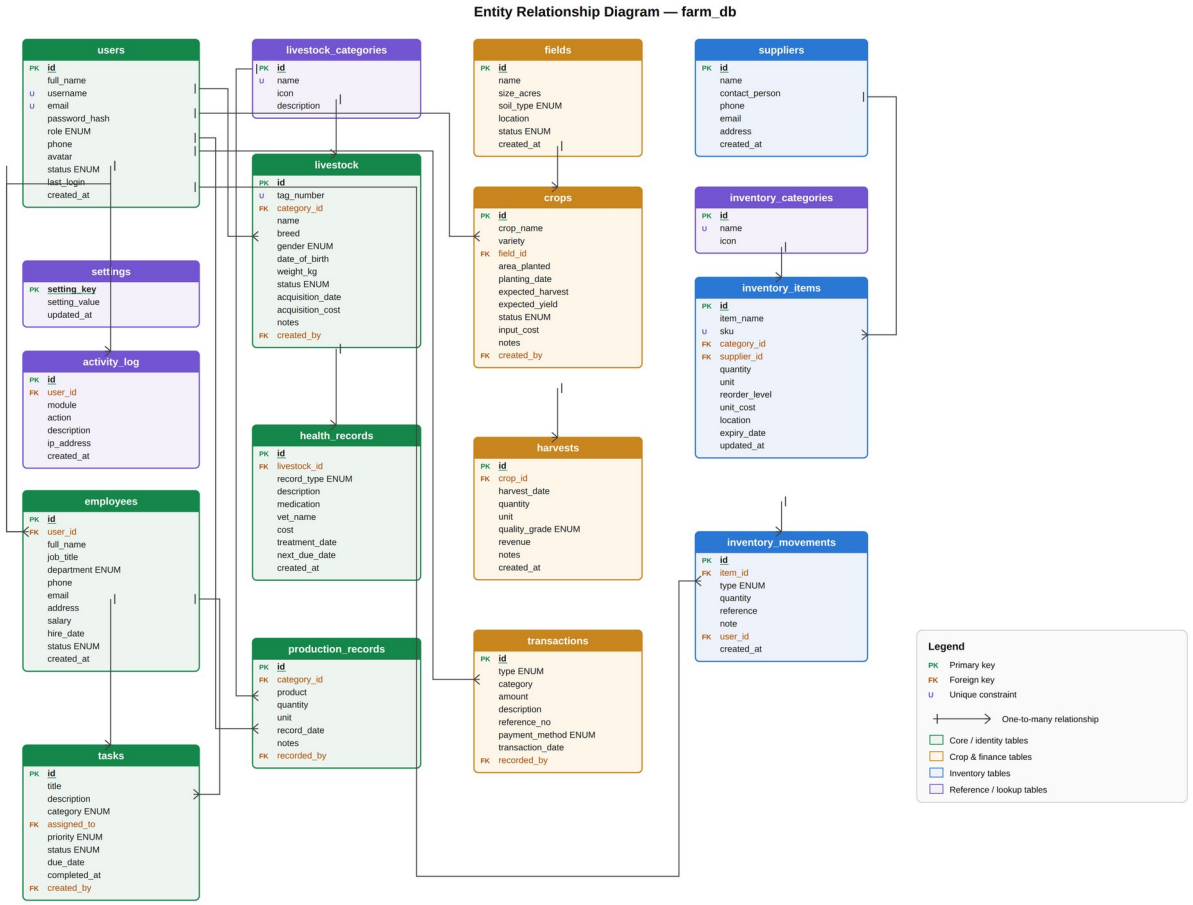

Plate A1 — Entity relationship diagram of the farm_db database (see Figure 3.3)

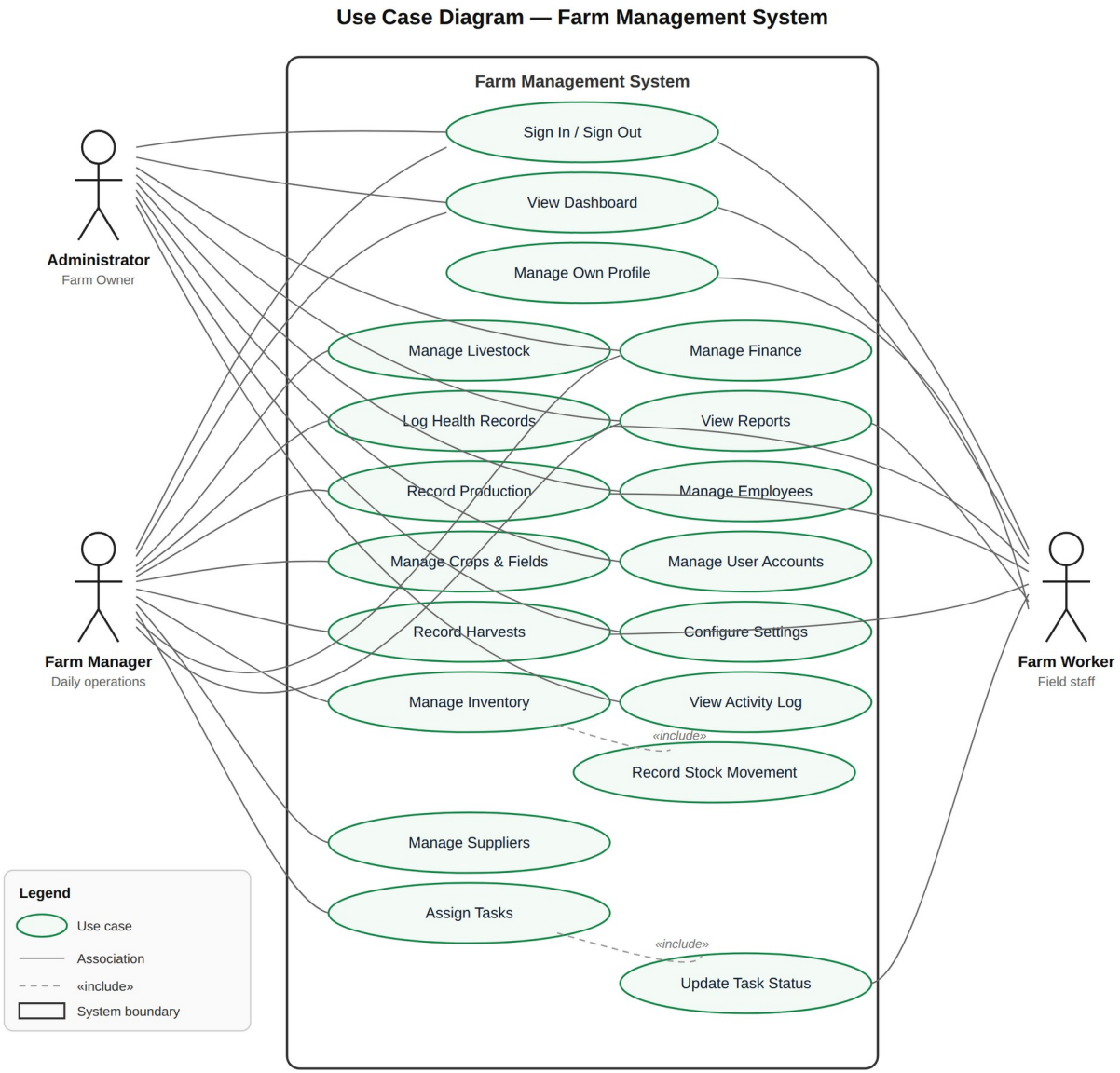

Plate A2 — Use case diagram (see Figure 3.2)

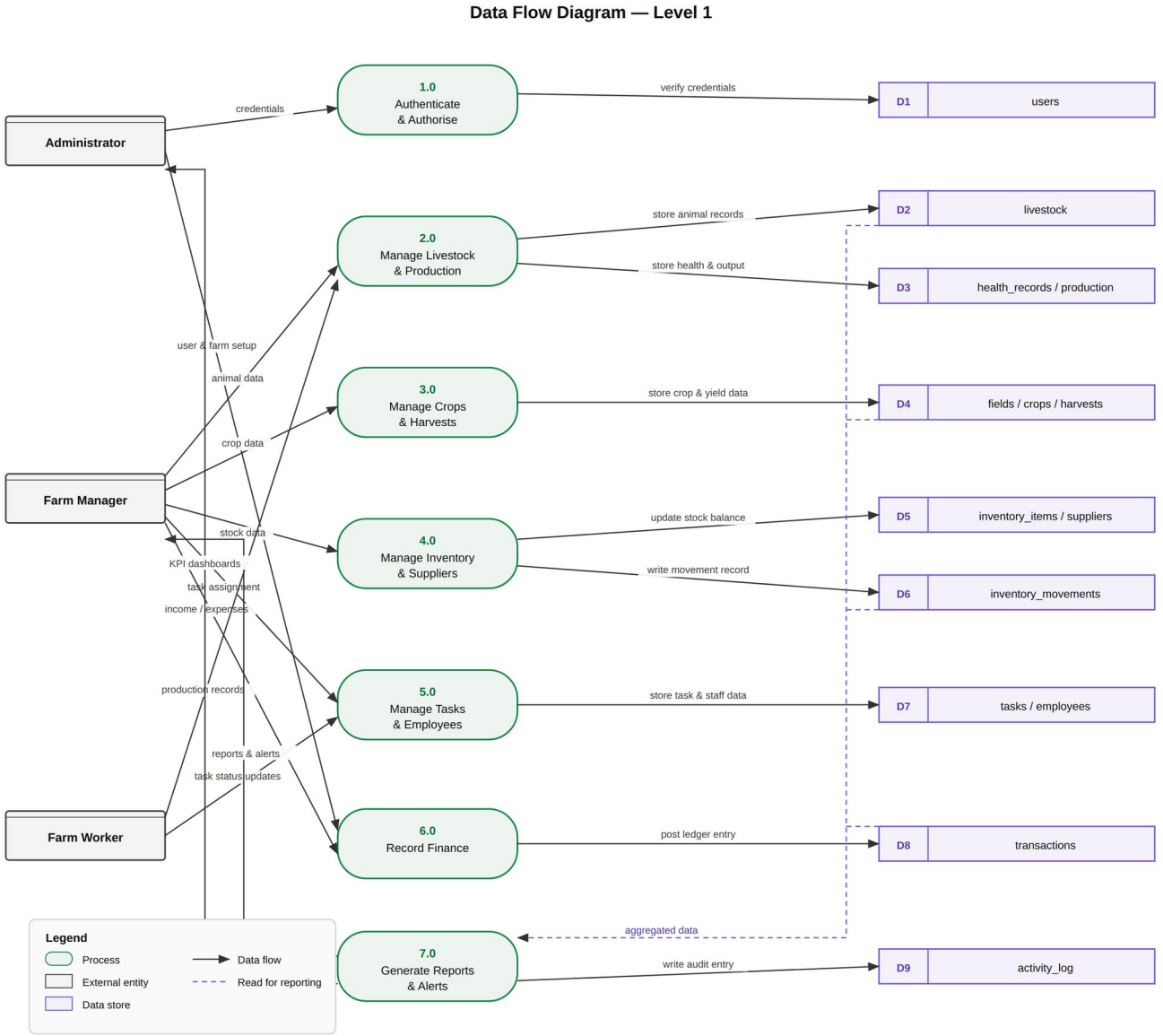

Plate A3 — Level 1 data flow diagram (see Figure 3.8)

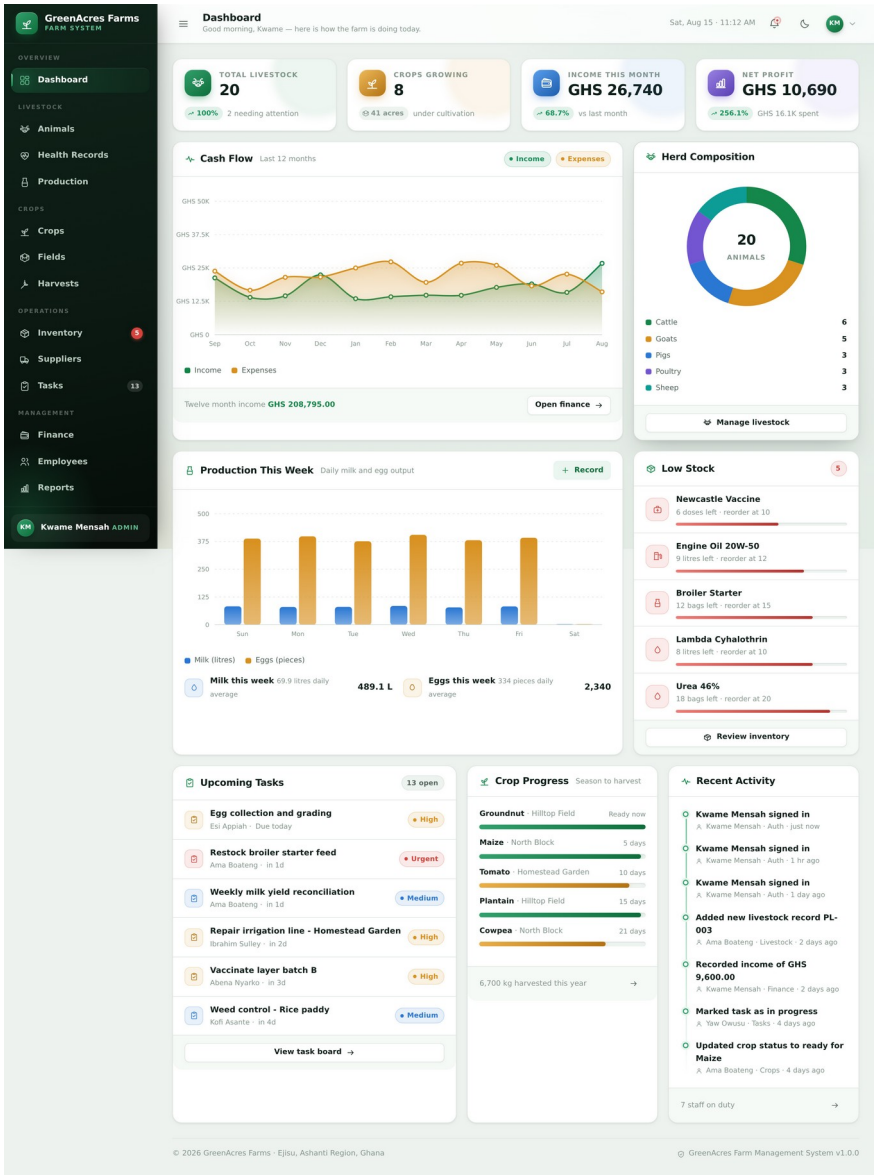

Plate A4 — Dashboard: complete page with all analytical panels

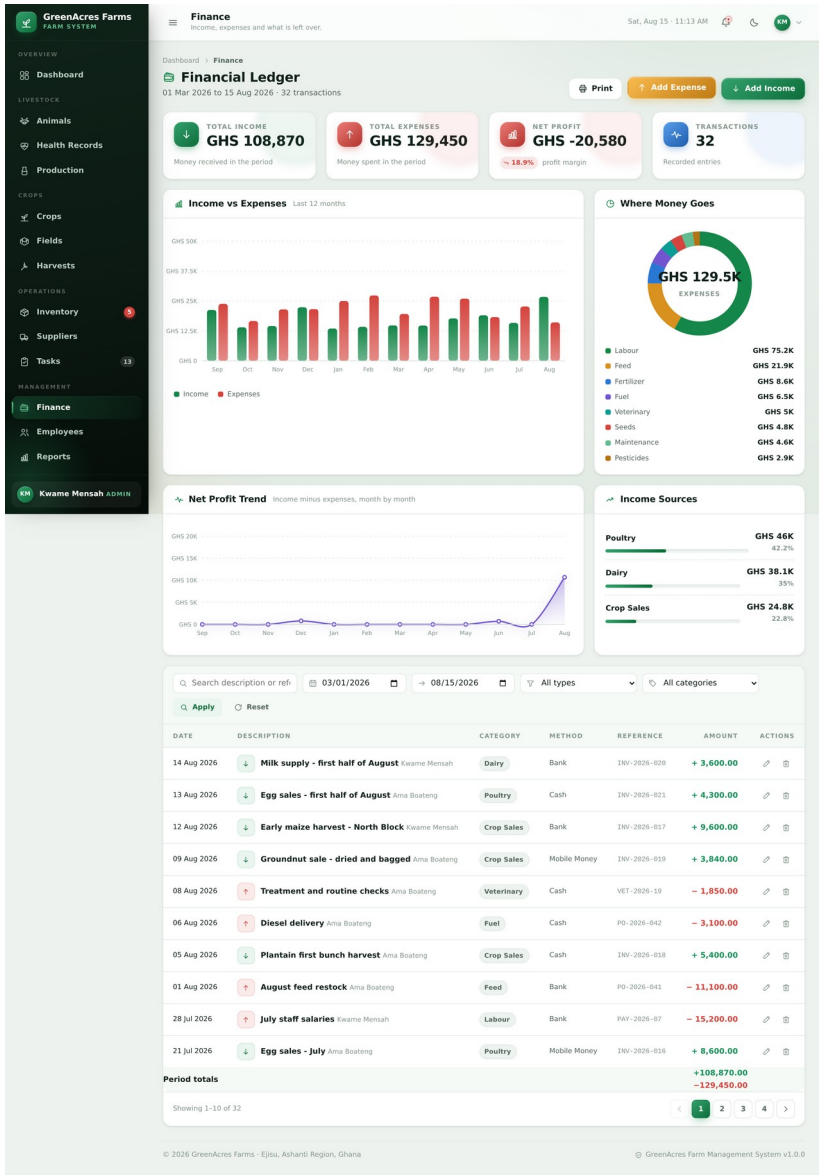

Plate A5 — Finance: complete page with profit trend and income sources

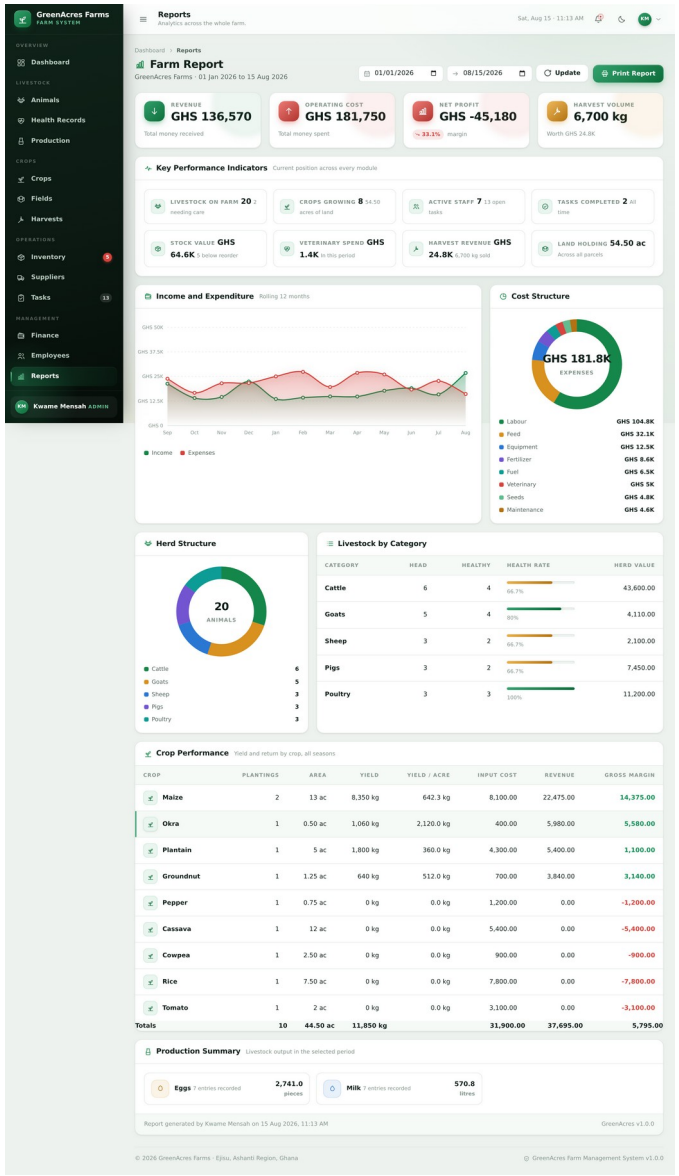

Plate A6 — Complete analytical report

APPENDIX B
DATABASE SCHEMA (EXTRACT)

The complete schema is contained in the file database/farm_db.sql accompanying this report. The extract below shows the definition of four representative tables, chosen to illustrate the use of primary keys, foreign keys, enumerated types, unique constraints, indexes and deletion rules.

B.1 users

```
CREATE TABLE `users` (
  `id`            INT AUTO_INCREMENT PRIMARY KEY,
  `full_name`     VARCHAR(120)  NOT NULL,
  `username`      VARCHAR(60)   NOT NULL UNIQUE,
  `email`         VARCHAR(140)  NOT NULL UNIQUE,
  `password_hash` VARCHAR(255)  NOT NULL,
  `role`          ENUM('admin','manager','worker') NOT NULL DEFAULT 'worker',
  `phone`         VARCHAR(30)   DEFAULT NULL,
  `avatar`        VARCHAR(180)  DEFAULT NULL,
  `status`        ENUM('active','suspended') NOT NULL DEFAULT 'active',
  `last_login`    DATETIME      DEFAULT NULL,
  `created_at`    DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX `idx_users_role` (`role`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

B.2 livestock

```
CREATE TABLE `livestock` (
  `id`               INT AUTO_INCREMENT PRIMARY KEY,
  `tag_number`       VARCHAR(40) NOT NULL UNIQUE,
  `category_id`      INT NOT NULL,
  `name`             VARCHAR(80)  DEFAULT NULL,
  `breed`            VARCHAR(80)  DEFAULT NULL,
  `gender`           ENUM('male','female') NOT NULL DEFAULT 'female',
  `date_of_birth`    DATE DEFAULT NULL,
  `weight_kg`        DECIMAL(8,2) NOT NULL DEFAULT 0,
  `status`           ENUM('healthy','sick','quarantine','pregnant',
                          'sold','deceased') NOT NULL DEFAULT 'healthy',
  `acquisition_date` DATE DEFAULT NULL,
  `acquisition_cost` DECIMAL(12,2) NOT NULL DEFAULT 0,
  `notes`            TEXT DEFAULT NULL,
  `created_by`       INT DEFAULT NULL,
  `created_at`       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT `fk_ls_cat`  FOREIGN KEY (`category_id`)
      REFERENCES `livestock_categories`(`id`) ON DELETE CASCADE,
  CONSTRAINT `fk_ls_user` FOREIGN KEY (`created_by`)
      REFERENCES `users`(`id`) ON DELETE SET NULL,
  INDEX `idx_ls_status` (`status`),
  INDEX `idx_ls_cat` (`category_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

B.3 inventory_items and inventory_movements

```
CREATE TABLE `inventory_items` (
  `id`            INT AUTO_INCREMENT PRIMARY KEY,
  `item_name`     VARCHAR(120) NOT NULL,
  `sku`           VARCHAR(40) NOT NULL UNIQUE,
  `category_id`   INT NOT NULL,
  `supplier_id`   INT DEFAULT NULL,
```

```
  `quantity`      DECIMAL(12,2) NOT NULL DEFAULT 0,
  `unit`          VARCHAR(20) NOT NULL,
  `reorder_level` DECIMAL(12,2) NOT NULL DEFAULT 0,
  `unit_cost`     DECIMAL(12,2) NOT NULL DEFAULT 0,
  `location`      VARCHAR(120) DEFAULT NULL,
  `expiry_date`   DATE DEFAULT NULL,
  `updated_at`    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
                    ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT `fk_inv_cat` FOREIGN KEY (`category_id`)
      REFERENCES `inventory_categories`(`id`) ON DELETE CASCADE,
  CONSTRAINT `fk_inv_sup` FOREIGN KEY (`supplier_id`)
      REFERENCES `suppliers`(`id`) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `inventory_movements` (
  `id`         INT AUTO_INCREMENT PRIMARY KEY,
  `item_id`    INT NOT NULL,
  `type`       ENUM('in','out','adjustment') NOT NULL,
  `quantity`   DECIMAL(12,2) NOT NULL,
  `reference`  VARCHAR(80) DEFAULT NULL,
  `note`       VARCHAR(200) DEFAULT NULL,
  `user_id`    INT DEFAULT NULL,
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT `fk_mv_item` FOREIGN KEY (`item_id`)
      REFERENCES `inventory_items`(`id`) ON DELETE CASCADE,
  CONSTRAINT `fk_mv_user` FOREIGN KEY (`user_id`)
      REFERENCES `users`(`id`) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

APPENDIX C
SELECTED SOURCE CODE

The complete source code accompanies this report. The extracts below are those referred to in the body of the text.

C.1 Authentication (includes/auth.php)

```
function attempt_login(string $identifier, string $password): ?string
{
    $user = one(
        'SELECT * FROM users WHERE username = ? OR email = ? LIMIT 1',
        [$identifier, $identifier]
    );

    // Same message either way — never reveal which accounts exist.
    if (!$user || !password_verify($password, $user['password_hash'])) {
        return 'Those details did not match any account.';
    }

    if ($user['status'] !== 'active') {
        return 'This account has been suspended.';
    }

    start_session();
    session_regenerate_id(true);   // guards against session fixation

    $_SESSION['user'] = [
        'id'   => (int) $user['id'],
        'role' => $user['role'],
        /* ... */
    ];

    update('users', ['last_login' => date('Y-m-d H:i:s')], (int) $user['id']);
    log_activity('auth', 'login', $user['full_name'] . ' signed in');

    return null;
}
```

C.2 Cross-site request forgery protection (includes/helpers.php)

```
function csrf_token(): string
{
    start_session();
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function csrf_verify(): void
{
    start_session();
    $sent = $_POST['csrf_token'] ?? '';

    // hash_equals compares in constant time, so the number of
    // matching leading bytes cannot be inferred from timing.
    if (!is_string($sent) || !hash_equals($_SESSION['csrf_token'] ?? '', $sent)) {
```

```
        http_response_code(419);
        exit('Security token expired or invalid.');
    }
}
```

APPENDIX D
INSTALLATION AND USER GUIDE

D.1 Installation

1. Install XAMPP with PHP 8.0 or newer from the Apache Friends website, accepting the default options.
2. Copy the project folder into the XAMPP web root: C:\xampp\htdocs on Windows, /Applications/XAMPP/htdocs on macOS, or /opt/lampp/htdocs on Linux.
3. Open the XAMPP control panel and start Apache and MySQL. Both indicators should turn green.
4. Open a browser and visit http://localhost/farm-management-system/install.php.
5. Confirm that all five environment checks pass, then select "Install now". The wizard creates the database, builds all tables and loads the demonstration data.
6. Delete install.php from the project folder. Leaving it in place would allow the database to be rebuilt and the data lost.
7. Visit http://localhost/farm-management-system/ and sign in.

Should the installer be unavailable, the database may instead be created by importing database/farm_db.sql through phpMyAdmin. The file creates the database itself, so no database need be created beforehand.

D.2 Demonstration Accounts

Role	Username	Password	Access
Administrator	admin	password123	Unrestricted, including user accounts and settings
Manager	manager	password123	Day-to-day operations and finance; no user administration
Worker	worker	password123	Records production, health and task status; read-only elsewhere

The demonstration passwords must be changed before the system holds real data.

D.3 Daily Operation

- Record production, such as milk and eggs, at the end of each working day.
- Record every stock receipt and issue at the time it happens, so that balances and alerts remain accurate.
- Record income and expenditure as it occurs rather than at month end.
- Check the notification bell each morning for low stock, tasks falling due and veterinary treatments due.
- Update the status of tasks as work is completed.

D.4 Keyboard Shortcuts

Key	Action
/	Move the cursor to the search field on the current page

Key	Action
Ctrl + K (Cmd + K on macOS)	Move the cursor to the search field and select its contents
n	Open the "new record" dialogue for the current module
Esc	Close the open dialogue

D.5 Troubleshooting

Symptom	Cause and remedy
"Database connection failed"	MySQL is not running. Start it from the XAMPP control panel.
"Unknown database farm_db"	The database has not been created. Run install.php or import the SQL file.
"Access denied for user root"	The MySQL root account has a password. Set DB_PASS in config/config.php.
Raw PHP code appears in the browser	The file was opened directly from disk. It must be served through Apache at a http://localhost address.
Page appears unstyled	The assets folder was not copied with the project.
Blank white page	A PHP error with DEBUG_MODE off. Set it to true temporarily to read the message.
Apache will not start	Another program is using port 80. Change the Apache port in XAMPP and use http://localhost:8080/ instead.

D.6 Backup Procedure

A backup should be taken weekly. In phpMyAdmin, select the farm_db database, choose the Export tab and select Go. The resulting SQL file is a complete copy of the database and should be stored on separate media, away from the machine running the system. To restore, create an empty database of the same name and import the file.